Contents

1 Basic

1.1 Default Code [81fabd]

```
#include <bits/stdc++.h>
#define For(i, a, b) for (int i = a; i <= b; i++)
#define Forr(i, a, b) for (int i = a; i >= b; i--)
#define F first
#define S second
#define fs F
#define sc S
#define iter(x) x.begin(), x.end()
#define sz(x) ((int)x.size())
using namespace std;
using ll = long long;
using pii = pair<int, int>;
using pll = pair<ll, ll>;

int32_t main() {
  ios::sync_with_stdio(false);
  cin.tie(0); // =^-w-^=
}
```

1.2 Debugger [0d7909]

```
#ifdef zisk
template<class T> void _d(T &&x) {
  if constexpr (ranges::range<T> &&
      !is_convertible_v<T, string_view>) {
    cerr << "{ "; for (auto &&i:x) _d(i); cerr << "} ";
  } else if constexpr (requires { get<0>(x); }) {
    cerr << "( ";
    apply([](auto &&...a){ (_d(a), ...); }, x);
    cerr << ") ";
  } else cerr << x << " ";
} // ranges::subrange(l, r)
void _db() { cerr << "\e[0m\n"; }
void _db(auto &&...a) {
    cerr << "\e[1;33m"; (_d(a), ...); _db(); }
#define debug(...) _db(#__VA_ARGS__, ":", __VA_ARGS__)
#define safe _db(__PRETTY_FUNCTION__,__LINE__,"safe")
#else
#define safe void()
#define debug(...) void()
#endif
```

1.3 Fast IO [604966]

```
#pragma GCC optimize("Ofast,inline,unroll-loops")
#pragma GCC target("bmi,bmi2,lzcnt,popcnt,avx2,sse,sse2
    ,sse3,ssse3,sse4,popcnt,abm,mmx,avx,tune=native")
#include<unistd.h>
char OB[65536]; int OP;
inline char RC() {
  static char buf[65536], *p = buf, *q = buf;
  return p == q && (q = (p = buf) + read(0, buf, 65536)
      ) == buf ? -1 : *p++;
}
inline int R() {
  static char c;
  while((c = RC()) < '0'); int a = c ^ '0';
  while((c = RC()) >= '0') a *= 10, a += c ^ '0';
  return a;
}
inline void W(int n) {
  static char buf[12], p;
  if (n == 0) OB[OP++]='0'; p = 0;
  while (n) buf[p++] = '0' + (n % 10), n /= 10;
  for (--p; p >= 0; --p) OB[OP++] = buf[p];
  if (OP > 65520) write(1, OB, OP), OP = 0;
}
```

1.4 vimrc [898839]

```
sy on
se nu rnu so=8 bs=2 sw=4 ts=4 sts=4 hls ls=2 cin acd
    mouse=a et
map <F9> :w<bar>!g++ "%" -o %:r -std=c++20 -Wall -
    Wextra -Wshadow -Wconversion -O2 -Dzisk -g -
    fsanitize=address,undefined<CR>
map <F8> :!./%:r<CR>
inoremap {<CR> {<CR>}<ESC>O
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space
    :]' \| md5sum \| cut -c-6
inoremap fj <ESC>
vnoremap fj <ESC>
" -D_GLIBCXX_ASSERTIONS, -D_GLIBCXX_DEBUG
```

2 Graph

2.1 2SAT (SCC) [1aebd0]

```
struct TwoSAT {
  // 0-indexed
  // idx i * 2 -> +i, i * 2 + 1 -> -i
  vector <vector <int>> adj, radj;
  vector <int> dfs_ord, idx, solution;
  vector <bool> vis;
  int n, nscc;
  TwoSAT () = default;
  TwoSAT (int _n) : n(_n), nscc(0) {
    adj.resize(n * 2), radj.resize(n * 2);
  }
  void add_clause(int x, int y) {
    // (x or y) = true
    int nx = x ^ 1, ny = y ^ 1;
```

```
    adj[nx].push_back(y), radj[y].push_back(nx);
    adj[ny].push_back(x), radj[x].push_back(ny);
  }
  void add_ifthen(int x, int y) {
    // if x = true then y = true
    add_clause(x ^ 1, y);
  }
  void add_must(int x) {
    // x = true
    int nx = x ^ 1;
    adj[nx].pb(x), radj[x].pb(nx);
  }
  void dfs(int v) {
    vis[v] = true;
    for (int u : adj[v]) if (!vis[u])
      dfs(u);
    dfs_ord.push_back(v);
  }
  void rdfs(int v) {
    idx[v] = nscc;
    for (int u : radj[v]) if (idx[u] == -1)
      rdfs(u);
  }
  bool find_sol() {
    vis.assign(n * 2, false), idx.assign(n * 2, -1),
        solution.assign(n, -1);
    for (int i = 0; i < n * 2; ++i) if (!vis[i])
      dfs(i);
    reverse(dfs_ord.begin(), dfs_ord.end());
    for (int i : dfs_ord) if (idx[i] == -1)
      rdfs(i), nscc++;
    for (int i = 0; i < n; i++) {
      if (idx[i << 1] == idx[i << 1 | 1])
        return false;
      if (idx[i << 1] < idx[i << 1 | 1])
        solution[i] = 0;
      else
        solution[i] = 1;
    }
    return true;
  }
};
```

2.2 VertexBCC [d04ebe]

```
struct BCC{ // 0-based, allow multi edges but not allow
     loops
  int n, m, cnt = 0;
  // n:|V|, m:|E|, cnt:#bcc
  // bcc i : vertices bcc_v[i] and edges bcc_e[i]
  vector<vector<int>> bcc_v, bcc_e;
  vector<vector<pii>> g; // original graph
  vector<pii> edges; // 0-based
  BCC(int _n, vector<pii> _edges):
    n(_n), m(SZ(_edges)), g(_n), edges(_edges){
      for(int i = 0; i < m; i++){
        auto [u, v] = edges[i];
        g[u].pb(pii(v, i)); g[v].pb(pii(u, i));
      }
    }
  void make_bcc(){ bcc_v.pb(); bcc_e.pb(); cnt++; }
  // modify these if you need more information
  void add_v(int v){ bcc_v.back().pb(v); }
  void add_e(int e){ bcc_e.back().pb(e); }
  void build(){
    vector<int> in(n, -1), low(n, -1), stk;
    vector<vector<int>> up(n);
    int ts = 0;
    auto _dfs = [&](auto dfs, int now, int par, int pe)
        -> void{
      if(pe != -1) up[now].pb(pe);
      in[now] = low[now] = ts++;
      stk.pb(now);
      for(auto [v, e] : g[now]){
        if(e == pe) continue;
        if(in[v] != -1){
          if(in[v] < in[now]) up[now].pb(e);
          low[now] = min(low[now], in[v]);
          continue;
        }
        dfs(dfs, v, now, e);
        low[now] = min(low[now], low[v]);
      }
      if((now != par && low[now] >= in[par]) || (now ==
          par && SZ(g[now]) == 0)){
        make_bcc();
        for(int v = stk.back();; v = stk.back()){
          stk.pop_back(), add_v(v);
          for(int e : up[v]) add_e(e);
          if(v == now) break;
        }
        if(now != par) add_v(par);
      }
    };
    for(int i = 0; i < n; i++)
      if(in[i] == -1) _dfs(_dfs, i, i, -1);
  }
};
```

2.3 EdgeBCC [8a9523]

```
vector <int> adj[N];
struct EdgeBCC {
  // 0-indexed
  vector <int> newadj[N];
  vector <int> low, dep, idx, stk, par;
  vector <bool> bridge; // edge i -> pa[i] is bridge ?
  int n, nbcc;
  EdgeBCC () = default;
  EdgeBCC (int _n) : n(_n), nbcc(0) {
    low.assign(n, -1), dep.assign(n, -1), idx.assign(n,
         -1);
    par.assign(n, -1), bridge.assign(n, false);
    for (int i = 0; i < n; ++i) if (dep[i] == -1) {
      dfs(i, -1);
    }
    for (int i = 1; i < n; ++i) if (bridge[i]) {
      newadj[idx[i]].pb(idx[par[i]]);
      newadj[idx[par[i]]].pb(idx[i]);
    }
  }
  void dfs(int v, int pa) {
    low[v] = dep[v] = ~pa ? dep[pa] + 1 : 0;
    par[v] = pa;
    stk.push_back(v);
    bool visp = false;
    for (int u : adj[v]) {
      if (!visp && u == pa) {
        visp = true;
      } else if (dep[u] == -1) {
        dfs(u, v);
        low[v] = min(low[v], low[u]);
      } else {
        low[v] = min(low[v], low[u]);
      }
    }
    if (low[v] == dep[v]) {
      if(~pa) bridge[v] = true;
      int x;
      do {
        x = stk.back(), stk.pop_back();
        idx[x] = nbcc;
      } while (x != v);
      nbcc++;
    }
  }
};
```

2.4 Centroid Decomposition [2356be]

```
vector <int> adj[N];
struct CentroidDecomposition {
  // 0-index
  vector <int> sz, cd_pa;
  int n;
  CentroidDecomposition () = default;
  CentroidDecomposition (int _n) : n(_n) {
    sz.assign(n, 0), cd_pa.assign(n, -2);
    dfs_cd(0, -1);
  }
  void dfs_sz(int v, int pa) {
    sz[v] = 1;
    for (int u : adj[v]) if (u != pa && cd_pa[u] == -2)
      dfs_sz(u, v), sz[v] += sz[u];
```

```cpp
  }
  int dfs_cen(int v, int pa, int s) {
    for (int u : adj[v]) if (u != pa && cd_pa[u] == -2) {
      if (sz[u] * 2 > s)
        return dfs_cen(u, v, s);
    }
    return v;
  }
  vector <int> block;
  void dfs_cd(int v, int pa) {
    dfs_sz(v, pa);
    int c = dfs_cen(v, pa, sz[v]);
    cd_pa[c] = pa;
    // centroid D&C
    for (int u : adj[c]) if (cd_pa[u] == -2) {
      dfs_ans(u, c);
      // do something
    }
    for (int u : adj[c]) if (cd_pa[u] == -2) {
      dfs_cd(u, c);
    }
  }
  void dfs_ans(int v, int pa) {
    // calculate path through centroid
    // do something
    // remember delete path from the same size
    for (int u : adj[v]) if (u != pa && cd_pa[u] == -2)
      dfs_ans(u, v);
  }
  // Centroid Tree Property:
  // let k = lca(u, v) in Centroid Tree, then dis(u, v) = dis(u, k) + dis(k, v)
};
```

2.5 Count Cycles [c7e8f2]

```cpp
// ord = sort by deg decreasing, rk[ord[i]] = i
// D[i] = edge point from rk small to rk big
for (int x : ord) { // c3
  for (int y : D[x]) vis[y] = 1;
  for (int y : D[x]) for (int z : D[y]) c3 += vis[z];
  for (int y : D[x]) vis[y] = 0;
}
for (int x : ord) { // c4
  for (int y : D[x]) for (int z : adj[y])
    if (rk[z] > rk[x]) c4 += vis[z]++;
  for (int y : D[x]) for (int z : adj[y])
    if (rk[z] > rk[x]) --vis[z];
} // both are O(M*sqrt(M)), test @ 2022 CCPC guangzhou
```

2.6 DirectedMST [d3eb5f]

```cpp
using D = int;
struct edge {
  int u, v; D w;
};
// 0-based, return index of edges
vector<int> dmst(vector<edge> &e, int n, int root) {
  using T = pair <D, int>;
  using PQ = pair <priority_queue <T, vector <T>, greater <T>>, D>;
  auto push = [](PQ &pq, T v) {
    pq.first.emplace(v.first - pq.second, v.second);
  };
  auto top = [](const PQ &pq) -> T {
    auto r = pq.first.top();
    return {r.first + pq.second, r.second};
  };
  auto join = [&push, &top](PQ &a, PQ &b) {
    if (a.first.size() < b.first.size()) swap(a, b);
    while (!b.first.empty())
      push(a, top(b)), b.first.pop();
  };
  vector<PQ> h(n * 2);
  for (int i = 0; i < e.size(); ++i)
    push(h[e[i].v], {e[i].w, i});
  vector<int> a(n * 2), v(n * 2, -1), pa(n * 2, -1), r(n * 2);
  iota(a.begin(),a.end(), 0);
  auto o = [&](int x) { int y;
    for (y = x; a[y] != y; y = a[y]);
    for (int ox = x; x != y; ox = x)
      x = a[x], a[ox] = y;
    return y;
  };
  v[root] = n + 1;
  int pc = n;
  for (int i = 0; i < n; ++i) if (v[i] == -1) {
    for (int p = i; v[p] == -1 || v[p] == i; p = o(e[r[p]].u)) {
      if (v[p] == i) {
        int q = p; p = pc++;
        do {
          h[q].second = -h[q].first.top().first;
          join(h[pa[q] = a[q] = p], h[q]);
        } while ((q = o(e[r[q]].u)) != p);
      }
      v[p] = i;
      while (!h[p].first.empty() && o(e[top(h[p]).second].u) == p)
        h[p].first.pop();
      r[p] = top(h[p]).second;
    }
  }
  vector<int> ans;
  for (int i = pc - 1; i >= 0; i--)
    if (i != root && v[i] != n) {
      for (int f = e[r[i]].v; f != -1 && v[f] != n; f = pa[f]) v[f] = n;
      ans.pb(r[i]);
    }
  return ans;
}
```

2.7 Dominator Tree [6378d5]

```cpp
struct Dominator_tree {
  int n, id;
  vector <vector <int>> adj, radj, bucket;
  vector <int> sdom, dom, vis, rev, par, rt, mn;
  Dominator_tree (int _n) : n(_n), id(0) {
    adj.resize(n), radj.resize(n), bucket.resize(n);
    sdom.resize(n), dom.resize(n, -1), vis.resize(n, -1);
    rev.resize(n), rt.resize(n), mn.resize(n), par.resize(n);
  }
  void add_edge(int u, int v) {adj[u].pb(v);}
  int query(int v, bool x) {
    if (rt[v] == v) return x ? -1 : v;
    int p = query(rt[v], true);
    if (p == -1) return x ? rt[v] : mn[v];
    if (sdom[mn[v]] > sdom[mn[rt[v]]]) mn[v] = mn[rt[v]];
    rt[v] = p;
    return x ? p : mn[v];
  }
  void dfs(int v) {
    vis[v] = id, rev[id] = v;
    rt[id] = mn[id] = sdom[id] = id, id++;
    for (int u : adj[v]) {
      if (vis[u] == -1) dfs(u), par[vis[u]] = vis[v];
      radj[vis[u]].pb(vis[v]);
    }
  }
  void build(int s) {
    dfs(s);
    for (int i = id - 1; ~i; --i) {
      for (int u : radj[i]) {
        sdom[i] = min(sdom[i], sdom[query(u, false)]);
      }
      if (i) bucket[sdom[i]].pb(i);
      for (int u : bucket[i]) {
        int p = query(u, false);
        dom[u] = sdom[p] == i ? i : p;
      }
      if (i) rt[i] = par[i];
    }
    vector <int> res(n, -1);
    for (int i = 1; i < id; ++i) {
      if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
    }
```

```
    for (int i = 1; i < id; ++i) res[rev[i]] = rev[dom[
        i]];
    res[s] = s;
    dom = res;
  } // dom[]: parent on dom. tree, -1 if not reachable
};
```

2.8 Heavy Light Decomposition [372958]

```
vector <int> adj[N];
struct HLD {
  // 0-index
  vector <int> dep, pt, hd, idx, sz, par, vis;
  int n, _t;
  HLD () = default;
  HLD (int _n) : n(_n) {
    pt.assign(n,-1), hd.assign(n,-1), par.assign(n,-1);
    idx.assign(n,0), sz.assign(n,0), dep.assign(n,0),
        vis.assign(n,0);
    _t = 0;
    for (int i = 0; i < n; ++i) if (!vis[i]) {
      dfs1(i, -1);
      dfs2(i, -1, 0);
    }
  }
  void dfs1(int v, int pa) {
    par[v] = pa;
    dep[v] = ~pa ? dep[pa] + 1 : 0;
    sz[v] = vis[v] = 1;
    for (int u : adj[v]) if (u != pa) {
      dfs1(u, v);
      if (pt[v] == -1 || sz[pt[v]] < sz[u])
        pt[v] = u;
      sz[v] += sz[u];
    }
  }
  void dfs2(int v, int pa, int h) {
    if (v == -1)
      return;
    idx[v] = _t++, hd[v] = h;
    dfs2(pt[v], v, h);
    for (int u : adj[v]) if (u != pa && u != pt[v]) {
      dfs2(u, v, u);
    }
  }
  void modify(int u, int v) {
    while (hd[u] != hd[v]) {
      if (dep[hd[u]] < dep[hd[v]])
        swap(u, v);
      // range [idx[hd[u]], idx[u] + 1)
      u = par[hd[u]];
    }
    if (dep[u] < dep[v])
      swap(u, v);
    // range [idx[v], idx[u] + 1)
  }
  int query(int u, int v) {
    int ans = 0;
    while (hd[u] != hd[v]) {
      if (dep[hd[u]] < dep[hd[v]])
        swap(u, v);
      // range [idx[hd[u]], idx[u] + 1)
      u = par[hd[u]];
    }
    if (dep[u] < dep[v])
      swap(u, v);
    // range [idx[v], idx[u] + 1)
    return ans;
  }
};
```

2.9 Virtual Tree [dcbe4f]

```
// need lca
vector <int> _g[N], stk;
int st[N], ed[N];
void solve(vector<int> v) {
  auto cmp = [&](int x, int y) {return st[x] < st[y];};
  sort(all(v), cmp);
  int sz = v.size();
  for (int i = 0; i < sz - 1; ++i)
    v.pb(lca(v[i], v[i + 1]));
  sort(all(v), cmp);
  v.resize(unique(all(v)) - v.begin());
  stk.clear(), stk.pb(v[0]);
  for (int i = 1; i < v.size(); ++i) {
    int x = v[i];
    while (ed[stk.back()] < ed[x]) stk.pop_back();
    _g[stk.back()].pb(x), stk.pb(x);
  }
  // do something
  for (int i : v) _g[i].clear();
}
```

2.10 Vizing [fa4b32]

```
namespace vizing { // returns edge coloring in adjacent
     matrix G. 1 - based
const int N = 105;
int C[N][N], G[N][N], X[N], vst[N], n;
void init(int _n) { n = _n;
  for (int i = 0; i <= n; ++i)
    for (int j = 0; j <= n; ++j)
      C[i][j] = G[i][j] = 0;
}
void solve(vector<pii> &E) {
  auto update = [&](int u)
  { for (X[u] = 1; C[u][X[u]]; ++X[u]); };
  auto color = [&](int u, int v, int c) {
    int p = G[u][v];
    G[u][v] = G[v][u] = c;
    C[u][c] = v, C[v][c] = u;
    C[u][p] = C[v][p] = 0;
    if (p) X[u] = X[v] = p;
    else update(u), update(v);
    return p;
  };
  auto flip = [&](int u, int c1, int c2) {
    int p = C[u][c1];
    swap(C[u][c1], C[u][c2]);
    if (p) G[u][p] = G[p][u] = c2;
    if (!C[u][c1]) X[u] = c1;
    if (!C[u][c2]) X[u] = c2;
    return p;
  };
  fill_n(X + 1, n, 1);
  for (int t = 0; t < E.size(); ++t) {
    int u = E[t].F, v0 = E[t].S, v = v0, c0 = X[u], c =
         c0, d;
    vector<pii> L;
    fill_n(vst + 1, n, 0);
    while (!G[u][v0]) {
      L.emplace_back(v, d = X[v]);
      if (!C[v][c]) for (int a = (int)L.size() - 1; a
          >= 0; --a) c = color(u, L[a].F, c);
      else if (!C[u][d]) for (int a = (int)L.size() -
          1; a >= 0; --a) color(u, L[a].F, L[a].S);
      else if (vst[d]) break;
      else vst[d] = 1, v = C[u][d];
    }
    if (!G[u][v0]) {
      for (; v; v = flip(v, c, d), swap(c, d));
      if (int a; C[u][c0]) {
        for (a = (int)L.size() - 2; a >= 0 && L[a].S !=
             c; --a);
        for (; a >= 0; --a) color(u, L[a].F, L[a].S);
      }
      else --t;
    }
  }
}
} // namespace vizing
```

2.11 Maximum Clique Dynamic [6dde09]

```
const int N = 150;
struct MaxClique { // Maximum Clique
  bitset<N> a[N], cs[N];
  int ans, sol[N], q, cur[N], d[N], n;
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; i++) a[i].reset();
  }
  void addEdge(int u, int v) { a[u][v] = a[v][u] = 1; }
```

```cpp
  void csort(vector<int> &r, vector<int> &c) {
    int mx = 1, km = max(ans - q + 1, 1), t = 0,
        m = r.size();
    cs[1].reset(), cs[2].reset();
    for (int i = 0; i < m; i++) {
      int p = r[i], k = 1;
      while ((cs[k] & a[p]).count()) k++;
      if (k > mx) mx++, cs[mx + 1].reset();
      cs[k][p] = 1;
      if (k < km) r[t++] = p;
    }
    c.resize(m);
    if (t) c[t - 1] = 0;
    for (int k = km; k <= mx; k++)
      for (int p = cs[k]._Find_first(); p < N;
           p = cs[k]._Find_next(p))
        r[t] = p, c[t] = k, t++;
  }
  void dfs(vector<int> &r, vector<int> &c, int l,
    bitset<N> mask) {
    while (!r.empty()) {
      int p = r.back();
      r.pop_back(), mask[p] = 0;
      if (q + c.back() <= ans) return;
      cur[q++] = p;
      vector<int> nr, nc;
      bitset<N> nmask = mask & a[p];
      for (int i : r)
        if (a[p][i]) nr.push_back(i);
      if (!nr.empty()) {
        if (l < 4) {
          for (int i : nr)
            d[i] = (a[i] & nmask).count();
          sort(nr.begin(), nr.end(),
            [&](int x, int y) { return d[x] > d[y]; });
        }
        csort(nr, nc), dfs(nr, nc, l + 1, nmask);
      } else if (q > ans) ans = q, copy_n(cur, q, sol);
      c.pop_back(), q--;
    }
  }
  int solve(bitset<N> mask = bitset<N>(
              string(N, '1'))) { // vertex mask
    vector<int> r, c;
    ans = q = 0;
    for (int i = 0; i < n; i++)
      if (mask[i]) r.push_back(i);
    for (int i = 0; i < n; i++)
      d[i] = (a[i] & mask).count();
    sort(r.begin(), r.end(),
      [&](int i, int j) { return d[i] > d[j]; });
    csort(r, c), dfs(r, c, 1, mask);
    return ans; // sol[0 ~ ans-1]
  }
} graph;
```

2.12 Minimum Steiner Tree [21acea]

```cpp
// O(V 3^T + V^2 2^T)
struct SteinerTree { // 0-base
  static const int T = 10, N = 105, INF = 1e9;
  int n, dst[N][N], dp[1 << T][N], tdst[N];
  int vcost[N]; // the cost of vertices
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) dst[i][j] = INF;
      dst[i][i] = vcost[i] = 0;
    }
  }
  void add_edge(int ui, int vi, int wi) {
    dst[ui][vi] = min(dst[ui][vi], wi);
  }
  void shortest_path() {
    for (int k = 0; k < n; ++k)
      for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
          dst[i][j] =
            min(dst[i][j], dst[i][k] + dst[k][j]);
  }
  int solve(const vector<int> &ter) {
    shortest_path();
    int t = SZ(ter);
    for (int i = 0; i < (1 << t); ++i)
      for (int j = 0; j < n; ++j) dp[i][j] = INF;
    for (int i = 0; i < n; ++i) dp[0][i] = vcost[i];
    for (int msk = 1; msk < (1 << t); ++msk) {
      if (!(msk & (msk - 1))) {
        int who = __lg(msk);
        for (int i = 0; i < n; ++i)
          dp[msk][i] =
            vcost[ter[who]] + dst[ter[who]][i];
      }
      for (int i = 0; i < n; ++i)
        for (int submsk = (msk - 1) & msk; submsk;
             submsk = (submsk - 1) & msk)
          dp[msk][i] = min(dp[msk][i],
            dp[submsk][i] + dp[msk ^ submsk][i] -
              vcost[i]);
      for (int i = 0; i < n; ++i) {
        tdst[i] = INF;
        for (int j = 0; j < n; ++j)
          tdst[i] =
            min(tdst[i], dp[msk][j] + dst[j][i]);
      }
      for (int i = 0; i < n; ++i) dp[msk][i] = tdst[i];
    }
    int ans = INF;
    for (int i = 0; i < n; ++i)
      ans = min(ans, dp[(1 << t) - 1][i]);
    return ans;
  }
};
```

2.13 Theory

$|\text{Maximum independent edge set}| = |V| - |\text{Minimum edge cover}|$

$|\text{Maximum independent set}| = |V| - |\text{Minimum vertex cover}|$

3 Data Structure

3.1 LiChao Tree [90f481]

```cpp
//C is range of x
//INF is big enough integer
struct Line {
  ll m,k;
  Line(ll _m=0,ll _k=0): m(_m),k(_k){}
  ll val(ll x){return m*x+k;}
};
struct LiChaoTree { //max y value
  Line st[C<<2];
  void init(int l,int r,int id) {
    st[id]=Line(0,0);
    if(l==r) return;
    int mid=(l+r)/2;
    init(l,mid,id<<1);
    init(mid+1,r,id<<1|1);
  }
  void upd(int l,int r,Line seg,int id) {
    if(l==r) {
      if(seg.val(l)>st[id].val(l)) st[id]=seg;
      return;
    }
    int mid=(l+r)/2;
    if(st[id].m>seg.m) swap(st[id],seg);
    if(st[id].val(mid)<seg.val(mid)) {
      swap(st[id],seg);
      upd(l,mid,seg,id<<1);
    } else upd(mid+1,r,seg,id<<1|1);
  }
  ll qry(int l,int r,ll x,int id) {
    if(l==r) return st[id].val(x);
    int mid=(l+r)/2;
    if(x<=mid) return max(qry(l,mid,x,id<<1),st[id].val
        (x));
    else return max(qry(mid+1,r,x,id<<1|1),st[id].val(x
        ));
  }
};
```

3.2 Dynamic Line Hull [8ec1c7]

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k;
      }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if(y == end()) return x->p = inf, 0;
    if(x->k == y->k) x->p = x -> m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while(isect(y, z)) z = erase(z);
    if(x != begin() && isect(--x, y)) isect(x, y =
        erase(y));
    while((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

3.3 Leftist Tree [473c12]

```
struct node {
  ll rk, data, sz, sum;
  node *l, *r;
  node(ll k) : rk(0), data(k), sz(1), l(0), r(0), sum(k
      ) {}
};
ll sz(node *p) { return p ? p->sz : 0; }
ll rk(node *p) { return p ? p->rk : -1; }
ll sum(node *p) { return p ? p->sum : 0; }
node *merge(node *a, node *b) {
  if (!a || !b) return a ? a : b;
  if (a->data < b->data) swap(a, b);
  a->r = merge(a->r, b);
  if (rk(a->r) > rk(a->l)) swap(a->r, a->l);
  a->rk = rk(a->r) + 1, a->sz = sz(a->l) + sz(a->r) +
      1;
  a->sum = sum(a->l) + sum(a->r) + a->data;
  return a;
}
void pop(node *&o) {
  node *tmp = o;
  o = merge(o->l, o->r);
  delete tmp;
}
```

3.4 Link Cut Tree [87ade4]

```
// weighted subtree size, weighted path max
struct LCT {
  int ch[N][2], pa[N], v[N], sz[N], sz2[N], w[N], mx[N
      ], _id;
  // sz := sum of v in splay, sz2 := sum of v in
      virtual subtree
  // mx := max w in splay
  bool rev[N];
  LCT() : _id(1) {}
  int newnode(int _v, int _w) {
    int x = _id++;
    ch[x][0] = ch[x][1] = pa[x] = 0;
    v[x] = sz[x] = _v;
    sz2[x] = 0;
    w[x] = mx[x] = _w;
    rev[x] = false;
    return x;
  }
  void pull(int i) {
    sz[i] = v[i] + sz2[i];
    mx[i] = w[i];
    if (ch[i][0])
      sz[i] += sz[ch[i][0]], mx[i] = max(mx[i], mx[ch[i
          ][0]]);
    if (ch[i][1])
      sz[i] += sz[ch[i][1]], mx[i] = max(mx[i], mx[ch[i
          ][1]]);
  }
  void push(int i) {
    if (rev[i]) reverse(ch[i][0]), reverse(ch[i][1]),
        rev[i] = false;
  }
  void reverse(int i) {
    if (!i) return;
    swap(ch[i][0], ch[i][1]);
    rev[i] ^= true;
  }
  bool isrt(int i) {// rt of splay
    if (!pa[i]) return true;
    return ch[pa[i]][0] != i && ch[pa[i]][1] != i;
  }
  void rotate(int i) {
    int p = pa[i], x = ch[p][1] == i, c = ch[i][!x], gp
        = pa[p];
    if (ch[gp][0] == p) ch[gp][0] = i;
    else if (ch[gp][1] == p) ch[gp][1] = i;
    pa[i] = gp, ch[i][!x] = p, pa[p] = i;
    ch[p][x] = c, pa[c] = p;
    pull(p), pull(i);
  }
  void splay(int i) {
    vector<int> anc;
    anc.push_back(i);
    while (!isrt(anc.back())) anc.push_back(pa[anc.back
        ()]);
    while (!anc.empty()) push(anc.back()), anc.pop_back
        ();
    while (!isrt(i)) {
      int p = pa[i];
      if (!isrt(p)) rotate(ch[p][1] == i ^ ch[pa[p]][1]
          == p ? i : p);
      rotate(i);
    }
  }
  void access(int i) {
    int last = 0;
    while (i) {
      splay(i);
      if (ch[i][1])
        sz2[i] += sz[ch[i][1]];
      sz2[i] -= sz[last];
      ch[i][1] = last;
      pull(i), last = i, i = pa[i];
    }
  }
  void makert(int i) {
    access(i), splay(i), reverse(i);
  }
  void link(int i, int j) {
    // assert(findrt(i) != findrt(j));
    makert(i);
    makert(j);
    pa[i] = j;
    sz2[j] += sz[i];
    pull(j);
  }
  void cut(int i, int j) {
    makert(i), access(j), splay(i);
    // assert(sz[i] == 2 && ch[i][1] == j);
    ch[i][1] = pa[j] = 0, pull(i);
  }
  int findrt(int i) {
    access(i), splay(i);
    while (ch[i][0]) push(i), i = ch[i][0];
    splay(i);
    return i;
  }
};
```

3.5 Splay Tree [e9029a]

```
struct Splay {
  int pa[N], ch[N][2], sz[N], rt, _id;
```

```
  ll v[N];
  Splay() {}
  void init() {
    rt = 0, pa[0] = ch[0][0] = ch[0][1] = -1;
    sz[0] = 1, v[0] = inf;
  }
  int newnode(int p, int x) {
    int id = _id++;
    v[id] = x, pa[id] = p;
    ch[id][0] = ch[id][1] = -1, sz[id] = 1;
    return id;
  }
  void rotate(int i) {
    int p = pa[i], x = ch[p][1] == i, gp = pa[p], c =
        ch[i][!x];
    sz[p] -= sz[i], sz[i] += sz[p];
    if (~c) sz[p] += sz[c], pa[c] = p;
    ch[p][x] = c, pa[p] = i;
    pa[i] = gp, ch[i][!x] = p;
    if (~gp) ch[gp][ch[gp][1] == p] = i;
  }
  void splay(int i) {
    while (~pa[i]) {
      int p = pa[i];
      if (~pa[p]) rotate(ch[pa[p]][1] == p ^ ch[p][1]
          == i ? i : p);
      rotate(i);
    }
    rt = i;
  }
  int lower_bound(int x) {
    int i = rt, last = -1;
    while (true) {
      if (v[i] == x) return splay(i), i;
      if (v[i] > x) {
        last = i;
        if (ch[i][0] == -1) break;
        i = ch[i][0];
      }
      else {
        if (ch[i][1] == -1) break;
        i = ch[i][1];
      }
    }
    splay(i);
    return last; // -1 if not found
  }
  void insert(int x) {
    int i = lower_bound(x);
    if (i == -1) {
      // assert(ch[rt][1] == -1);
      int id = newnode(rt, x);
      ch[rt][1] = id, ++sz[rt];
      splay(id);
    }
    else if (v[i] != x) {
      splay(i);
      int id = newnode(rt, x), c = ch[rt][0];
      ch[rt][0] = id;
      ch[id][0] = c;
      if (~c) pa[c] = id, sz[id] += sz[c];
      ++sz[rt];
      splay(id);
    }
  }
};
```

3.6 Treap [9dc91b]

```
struct Treap {
  int pri, sz, val;
  Treap *tl, *tr;
  Treap (int x) : val(x), sz(1), pri(rand()), tl(NULL),
      tr(NULL) {}
  void pull() {
    sz = (tl ? tl->sz : 0) + 1 + (tr ? tr->sz : 0);
  }
  void out() {
    if (tl) tl->out();
    cout << val << ' ';
    if (tr) tr->out();
  }
};
void print(Treap *t) {
  t->out();
  cout << endl;
}
Treap* merge(Treap *a, Treap *b) {
  if (!a || !b) return a ? a : b;
  if (a->pri < b->pri) {
    a->tr = merge(a->tr, b);
    a->pull();
    return a;
  } else {
    b->tl = merge(a, b->tl);
    b->pull();
    return b;
  }
}
void split(Treap* t, int k, Treap* &a, Treap* &b) {
  if (!t) a = b = NULL;
  else if ((t->tl ? t->tl->sz : 0) + 1 <= k) {
    a = t;
    split(t->tr, k - (t->tl ? t->tl->sz : 0) - 1, a->tr
        , b);
    a->pull();
  } else {
    b = t;
    split(t->tl, k, a, b->tl);
    b->pull();
  }
}
```

4 Flow/Matching

4.1 Hopcroft Karp [4b930f]

```
struct HopcroftKarp {
  const int INF = 1 << 30;
  vector<int> adj[N];
  int match[N], dis[N], v, n, m;
  bool matched[N], vis[N];
  bool dfs(int x) { // dffdce
    vis[x] = true;
    for (int y : adj[x])
      if (match[y] == -1 || (dis[match[y]] == dis[x] +
          1 && !vis[match[y]] && dfs(match[y]))) {
        match[y] = x, matched[x] = true;
        return true;
      }
    return false;
  }
  bool bfs() { // c444dc
    memset(dis, -1, sizeof(int) * n);
    queue<int> q;
    for (int x = 0; x < n; ++x) if (!matched[x])
      dis[x] = 0, q.push(x);
    int mx = INF;
    while (!q.empty()) {
      int x = q.front(); q.pop();
      for (int y : adj[x]) {
        if (match[y] == -1) {
          mx = dis[x];
          break;
        } else if (dis[match[y]] == -1)
          dis[match[y]] = dis[x] + 1, q.push(match[y]);
      }
    }
    return mx < INF;
  }
  int solve() { // 39e2af
    int res = 0;
    memset(match, -1, sizeof(int) * m);
    memset(matched, 0, sizeof(bool) * n);
    while (bfs()) {
      memset(vis, 0, sizeof(bool) * n);
      for (int x = 0; x < n; ++x) if (!matched[x])
        res += dfs(x);
    }
    return res;
  }
  void init(int _n, int _m) {
    n = _n, m = _m;
```

```
    for (int i = 0; i < n; ++i) adj[i].clear();
  }
  void add_edge(int x, int y) {
    adj[x].pb(y);
  }
};
```

4.2 Dinic [aa8522]

```
template <typename T> // 0-based, O(V^2E), unit flow: O(min(V^{2/3}E, E^{3/2})), bipartite matching: O(sqrt(V)E)
struct Dinic { // 0-based
  const T INF = numeric_limits<T>::max() / 2;
  struct edge { int to, rev; T cap, flow; };
  int n, s, t;
  vector <vector <edge>> g;
  vector <int> dis, cur;
  T dfs(int u, T cap) { // 09625c
    if (u == t || !cap) return cap;
    for (int &i = cur[u]; i < (int)g[u].size(); ++i) {
      edge &e = g[u][i];
      if (dis[e.to] == dis[u] + 1 && e.flow != e.cap) {
        T df = dfs(e.to, min(e.cap - e.flow, cap));
        if (df) {
          e.flow += df;
          g[e.to][e.rev].flow -= df;
          return df;
    } } }
    dis[u] = -1;
    return 0;
  }
  bool bfs() { // b06b3d
    fill(all(dis), -1);
    queue<int> q;
    q.push(s); dis[s] = 0;
    while (!q.empty()) {
      int v = q.front(); q.pop();
      for (auto &u : g[v])
        if (dis[u.to] == -1 && u.flow != u.cap) {
          q.push(u.to);
          dis[u.to] = dis[v] + 1;
        }
    }
    return dis[t] != -1;
  }
  T solve(int _s, int _t) { // 864a07
    s = _s; t = _t;
    T flow = 0, df;
    while (bfs()) {
      fill(all(cur), 0);
      while ((df = dfs(s, INF))) flow += df;
    }
    return flow;
  }
  void reset() {
    for (int i = 0; i < n; ++i)
      for (auto &j : g[i]) j.flow = 0;
  }
  void add_edge(int u, int v, T cap) {
    g[u].pb(edge{v, (int)g[v].size(), cap, 0});
    g[v].pb(edge{u, (int)g[u].size() - 1, 0, 0});
  //change g[v] to cap for undirected graphs
  }
  Dinic (int _n) : n(_n), g(n), dis(n), cur(n) {}
};
```

4.3 Min Cost Max Flow [e18ab8]

```
struct MCMF {
  const int INF = 1 << 30;
  struct edge {
    int v, f, c;
    edge (int _v, int _f, int _c) : v(_v), f(_f), c(_c) {}
  };
  vector <edge> E;
  vector <vector <int>> adj;
  vector <int> dis, pot, rt;
  int n, s, t;
  MCMF (int _n, int _s, int _t) : n(_n), s(_s), t(_t) {
    adj.resize(n);
  }
  void add_edge(int u, int v, int f, int c) {
    adj[u].pb(E.size()), E.pb(edge(v, f, c));
    adj[v].pb(E.size()), E.pb(edge(u, 0, -c));
  }
  bool SPFA() { // e9eac4
    rt.assign(n, -1), dis.assign(n, INF);
    vector <bool> vis(n, false);
    queue <int> q;
    q.push(s), dis[s] = 0, vis[s] = true;
    while (!q.empty()) {
      int v = q.front(); q.pop();
      vis[v] = false;
      for (int id : adj[v]) if (E[id].f > 0 && dis[E[id].v] > dis[v] + E[id].c + pot[v] - pot[E[id].v]) {
        dis[E[id].v] = dis[v] + E[id].c + pot[v] - pot[E[id].v], rt[E[id].v] = id;
        if (!vis[E[id].v]) vis[E[id].v] = true, q.push(E[id].v);
      }
    }
    return dis[t] != INF;
  }
  bool dijkstra() { // ce8d3f
    rt.assign(n, -1), dis.assign(n, INF);
    priority_queue <pair <int, int>, vector <pair <int, int>>, greater <pair <int, int>>> pq;
    dis[s] = 0, pq.emplace(dis[s], s);
    while (!pq.empty()) {
      int d, v; tie(d, v) = pq.top(); pq.pop();
      if (dis[v] < d) continue;
      for (int id : adj[v]) if (E[id].f > 0 && dis[E[id].v] > dis[v] + E[id].c + pot[v] - pot[E[id].v]) {
        dis[E[id].v] = dis[v] + E[id].c + pot[v] - pot[E[id].v], rt[E[id].v] = id;
        pq.emplace(dis[E[id].v], E[id].v);
      }
    }
    return dis[t] != INF;
  }
  pair <int, int> runFlow() { // 9d16df
    pot.assign(n, 0);
    int cost = 0, flow = 0;
    bool fr = true;
    while ((fr ? SPFA() : dijkstra())) {
      for (int i = 0; i < n; i++) {
        dis[i] += pot[i] - pot[s];
      }
      int add = INF;
      for (int i = t; i != s; i = E[rt[i] ^ 1].v) {
        add = min(add, E[rt[i]].f);
      }
      for (int i = t; i != s; i = E[rt[i] ^ 1].v) {
        E[rt[i]].f -= add, E[rt[i] ^ 1].f += add;
      }
      flow += add, cost += add * dis[t];
      fr = false;
      swap(dis, pot);
    }
    return make_pair(flow, cost);
  }
};
```

4.4 Min Cost Circulation [ea0477]

```
template <typename F, typename C>
struct MinCostCirculation {
  struct ep { int to; F flow; C cost; };
  int n; vector<int> vis; int visc;
  vector<int> fa, fae; vector<vector<int>> g;
  vector<ep> e; vector<C> pi;
  MinCostCirculation(int n_) : n(n_), vis(n), visc(0), g(n), pi(n) {}
  void add_edge(int u, int v, F fl, C cs) {
    g[u].emplace_back((int)e.size());
    e.emplace_back(v, fl, cs);
    g[v].emplace_back((int)e.size());
    e.emplace_back(u, 0, -cs);
  }
  C phi(int x) {
```

```
    if (fa[x] == -1) return 0;
    if (vis[x] == visc) return pi[x];
    vis[x] = visc;
    return pi[x] = phi(fa[x]) - e[fae[x]].cost;
  }
  int lca(int u, int v) {
    for (; u != -1 || v != -1; swap(u, v)) if (u != -1)
        {
      if (vis[u] == visc) return u;
      vis[u] = visc; u = fa[u];
    }
    return -1;
  }
  void pushflow(int x, C &cost) {
    int v = e[x ^ 1].to, u = e[x].to; ++visc;
    if (int w = lca(u, v); w == -1) {
      while (v != -1)
        swap(x ^= 1, fae[v]), swap(u, fa[v]), swap(u, v
            );
    } else {
      int z = u, dir = 0; F f = e[x].flow;
      vector<int> cyc = {x};
      for (int d : {0, 1})
        for (int i = (d ? u : v); i != w; i = fa[i]) {
          cyc.push_back(fae[i] ^ d);
          if (chmin(f, e[fae[i] ^ d].flow)) z = i, dir
              = d;
        }
      for (int i : cyc) {
        e[i].flow -= f; e[i ^ 1].flow += f;
        cost += f * e[i].cost;
      }
      if (dir) x ^= 1, swap(u, v);
      while (u != z)
        swap(x ^= 1, fae[v]), swap(u, fa[v]), swap(u, v
            );
    }
  }
  void dfs(int u) {
    vis[u] = visc;
    for (int i : g[u])
      if (int v = e[i].to; vis[v] != visc and e[i].flow
          )
        fa[v] = u, fae[v] = i, dfs(v);
  }
  C simplex() {
    fa.assign(g.size(), -1); fae.assign(g.size(), -1);
    C cost = 0; ++visc; dfs(0);
    for (int fail = 0; fail < ssize(e); )
      for (int i = 0; i < ssize(e); i++)
        if (e[i].flow and e[i].cost < phi(e[i ^ 1].to)
            - phi(e[i].to))
          fail = 0, pushflow(i, cost), ++visc;
        else ++fail;
    return cost;
  }
};
```

4.5 Kuhn Munkres [a138de]

```
template <typename T>
struct KM { // 0-based
  const T INF = 1 << 30;
  T w[N][N], hl[N], hr[N], slk[N];
  int fl[N], fr[N], pre[N], n;
  bool vl[N], vr[N];
  queue <int> q;
  KM () {}
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; ++i)
      for (int j = 0; j < n; ++j) w[i][j] = -INF;
  }
  void add_edge(int a, int b, T wei) { w[a][b] = wei; }
  bool check(int x) {
    if (vl[x] = 1, ~fl[x])
      return q.push(fl[x]), vr[fl[x]] = 1;
    while (~x) swap(x, fr[fl[x] = pre[x]]);
    return 0;
  }
  void bfs(int s) {
    fill(slk, slk + n, INF), fill(vl, vl + n, 0);
    fill(vr, vr + n, 0);
    while (!q.empty()) q.pop();
    q.push(s), vr[s] = 1;
    while (true) {
      T d;
      while (!q.empty()) {
        int y = q.front(); q.pop();
        for (int x = 0; x < n; ++x)
          if (!vl[x] && slk[x] >= (d = hl[x] + hr[y] -
              w[x][y]))
            if (pre[x] = y, d) slk[x] = d;
            else if (!check(x)) return;
      }
      d = INF;
      for (int x = 0; x < n; ++x)
        if (!vl[x] && d > slk[x]) d = slk[x];
      for (int x = 0; x < n; ++x) {
        if (vl[x]) hl[x] += d;
        else slk[x] -= d;
        if (vr[x]) hr[x] -= d;
      }
      for (int x = 0; x < n; ++x)
        if (!vl[x] && !slk[x] && !check(x)) return;
    }
  }
  T solve() {
    fill(fl, fl + n, -1), fill(fr, fr + n, -1);
    fill(hr, hr + n, 0);
    for (int i = 0; i < n; ++i)
      hl[i] = *max_element(w[i], w[i] + n);
    for (int i = 0; i < n; ++i) bfs(i);
    T res = 0;
    for (int i = 0; i < n; ++i) res += w[i][fl[i]];
    return res;
  }
};
```

4.6 Stoer Wagner (Min-cut) [ac255a]

```
struct SW {
  int g[N][N], sum[N], n;
  bool vis[N], dead[N];
  void init(int _n) {
    n = _n;
    for (int i = 0; i < n; ++i) fill(g[i], g[i] + n, 0)
        ;
    fill(dead, dead + n, false);
  }
  void add_edge(int u, int v, int w) {
    g[u][v] += w, g[v][u] += w;
  }
  int run() {
    int ans = 1 << 30;
    for (int round = 0; round + 1 < n; ++round) {
      fill(vis, vis + n, false), fill(sum, sum + n, 0);
      int num = 0, s = -1, t = -1;
      while (num < n - round) {
        int now = -1;
        for (int i = 0; i < n; ++i) if (!vis[i] && !
            dead[i]) {
          if (now == -1 || sum[now] < sum[i]) now = i;
        }
        s = t, t = now;
        vis[now] = true, num++;
        for (int i = 0; i < n; ++i) if (!vis[i] && !
            dead[i]) {
          sum[i] += g[now][i];
        }
      }
      ans = min(ans, sum[t]);
      for (int i = 0; i < n; ++i) {
        g[i][s] += g[i][t];
        g[s][i] += g[t][i];
      }
      dead[t] = true;
    }
    return ans;
  }
};
```

4.7 GomoryHu Tree [90ead2]

```cpp
vector <array <int, 3>> GomoryHu(Dinic <int> flow) {
  // Tree edge min = mincut (0-based)
  int n = flow.n;
  vector <array <int, 3>> ans;
  vector <int> rt(n);
  for (int i = 1; i < n; ++i) {
    int t = rt[i];
    flow.reset();
    ans.pb({i, t, flow.solve(i, t)});
    flow.bfs();
    for (int j = i + 1; j < n; ++j)
      if (rt[j] == t && flow.dis[j] != -1) rt[j] = i;
  }
  return ans;
}
```

4.8 General Graph Matching [2b7f20]

```cpp
struct Matching { // 0-based
  int fa[N], pre[N], match[N], s[N], v[N], n, tk;
  vector <int> g[N];
  queue <int> q;
  int Find(int u) {
    return u == fa[u] ? u : fa[u] = Find(fa[u]);
  }
  int lca(int x, int y) {
    tk++;
    x = Find(x), y = Find(y);
    for (; ; swap(x, y)) {
      if (x != n) {
        if (v[x] == tk) return x;
        v[x] = tk;
        x = Find(pre[match[x]]);
      }
    }
  }
  void blossom(int x, int y, int l) {
    while (Find(x) != l) {
      pre[x] = y, y = match[x];
      if (s[y] == 1) q.push(y), s[y] = 0;
      if (fa[x] == x) fa[x] = l;
      if (fa[y] == y) fa[y] = l;
      x = pre[y];
    }
  }
  bool bfs(int r) {
    for (int i = 0; i <= n; ++i) fa[i] = i, s[i] = -1;
    while (!q.empty()) q.pop();
    q.push(r);
    s[r] = 0;
    while (!q.empty()) {
      int x = q.front(); q.pop();
      for (int u : g[x]) {
        if (s[u] == -1) {
          pre[u] = x, s[u] = 1;
          if (match[u] == n) {
            for (int a = u, b = x, last; b != n; a =
                last, b = pre[a])
              last = match[b], match[b] = a, match[a] =
                  b;
            return true;
          }
          q.push(match[u]);
          s[match[u]] = 0;
        } else if (!s[u] && Find(u) != Find(x)) {
          int l = lca(u, x);
          blossom(x, u, l);
          blossom(u, x, l);
        }
      }
    }
    return false;
  }
  int solve() {
    int res = 0;
    for (int x = 0; x < n; ++x) {
      if (match[x] == n) res += bfs(x);
    }
    return res;
  }
  void init(int _n) {
    n = _n, tk = 0;
    for (int i = 0; i <= n; ++i) match[i] = pre[i] = n;
    for (int i = 0; i < n; ++i) g[i].clear(), v[i] = 0;
  }
  void add_edge(int u, int v) {
    g[u].push_back(v), g[v].push_back(u);
  }
};
```

4.9 Flow notes

- Bipartite Matching Restore Answer

  `runBfs();` Answer is $\{!vis[x] | x \in L\} \cup \{vis[x] | x \in R\}$
- Bipartite Minimum Weight Vertex Covering

  $S \to \{x | x \in L\}$, cap = weight of vertex x
  $\{x | x \in L\} \to \{y | y \in R\}$, cap = ∞
  $\{y | y \in R\} \to T$, cap = weight of vertex y

  For general version, change Dinic to MCMF and:

  $S \to \{x | x \in L\}$, cap = weight of vertex x, cost = 0
  $\{x | x \in L\} \to \{y | y \in R\}$, cap = ∞, cost = $-w$
  $\{y | y \in R\} \to T$, cap = weight of vertex y, cost = 0
- Useful Lemma

  (Bipartite Maximum Weight Independent Set) +
  (Bipartite Minimum Weight Vertex Covering) = weight sum
- Min Cut Model

  choose A but not choose B cost x: $A \to B$, cap = x
  choose A cost x: $A \to T$, cap = x
  not choose A cost x: $S \to A$, cap = x
  choose A gain $x \implies$ not choose A cost x, $tot+=x$
  choose A and choose B cost x: NO!!!
  Bipartite: can flip one side
- Min Cut Restore Answer

  runBfs(); Answer is $\{vis[x] | x \in V\}$

- Maximum/Minimum flow with lower bound / Circulation problem
  1. Construct super source S and sink T.
  2. For each edge (x, y, l, u), connect $x \to y$ with capacity $u - l$.
  3. For each vertex v, denote by $in(v)$ the difference between the sum of incoming lower bounds and the sum of outgoing lower bounds.
  4. If $in(v) > 0$, connect $S \to v$ with capacity $in(v)$, otherwise, connect $v \to T$ with capacity $-in(v)$.
     - To maximize, connect $t \to s$ with capacity ∞ (skip this in circulation problem), and let f be the maximum flow from S to T. If $f \neq \sum_{v \in V, in(v) > 0} in(v)$, there's no solution. Otherwise, the maximum flow from s to t is the answer.
     - To minimize, let f be the maximum flow from S to T. Connect $t \to s$ with capacity ∞ and let the flow from S to T be f'. If $f + f' \neq \sum_{v \in V, in(v) > 0} in(v)$, there's no solution. Otherwise, f' is the answer.
  5. The solution of each edge e is $l_e + f_e$, where f_e corresponds to the flow of edge e on the graph.

- Construct minimum vertex cover from maximum matching M on bipartite graph (X, Y)
  1. Redirect every edge: $y \to x$ if $(x, y) \in M$, $x \to y$ otherwise.
  2. DFS from unmatched vertices in X.
  3. $x \in X$ is chosen iff x is unvisited.
  4. $y \in Y$ is chosen iff y is visited.

- Minimum cost cyclic flow
  1. Consruct super source S and sink T
  2. For each edge (x, y, c), connect $x \to y$ with $(cost, cap) = (c, 1)$ if $c > 0$, otherwise connect $y \to x$ with $(cost, cap) = (-c, 1)$
  3. For each edge with $c < 0$, sum these cost as K, then increase $d(y)$ by 1, decrease $d(x)$ by 1
  4. For each vertex v with $d(v) > 0$, connect $S \to v$ with $(cost, cap) = (0, d(v))$
  5. For each vertex v with $d(v) < 0$, connect $v \to T$ with $(cost, cap) = (0, -d(v))$
  6. Flow from S to T, the answer is the cost of the flow $C + K$

- Maximum density induced subgraph
  1. Binary search on answer, suppose we're checking answer T
  2. Construct a max flow model, let K be the sum of all weights
  3. Connect source $s \to v, v \in G$ with capacity K
  4. For each edge (u, v, w) in G, connect $u \to v$ and $v \to u$ with capacity w
  5. For $v \in G$, connect it with sink $v \to t$ with capacity $K + 2T - (\sum_{e \in E(v)} w(e)) - 2w(v)$
  6. T is a valid answer if the maximum flow $f < K|V|$

- Minimum weight edge cover
  1. For each $v \in V$ create a copy v', and connect $u' \to v'$ with weight $w(u, v)$.
  2. Connect $v \to v'$ with weight $2\mu(v)$, where $\mu(v)$ is the cost of the cheapest edge incident to v.
  3. Find the minimum weight perfect matching on G'.

- Project selection problem
  1. If $p_v > 0$, create edge (s, v) with capacity p_v; otherwise, create edge (v, t) with capacity $-p_v$.
  2. Create edge (u, v) with capacity w with w being the cost of choosing u without choosing v.
  3. The mincut is equivalent to the maximum profit of a subset of projects.

5 String

5.1 AC Automaton [b9fe7c]

```
struct AC {
  int ch[N][26], to[N][26], fail[N], sz;
  vector <int> g[N];
  int cnt[N];
  AC () {sz = 0, extend();}
  void extend() {fill(ch[sz], ch[sz] + 26, 0), sz++;}
  int nxt(int u, int v) {
    if (!ch[u][v]) ch[u][v] = sz, extend();
    return ch[u][v];
  }
  int insert(string s) {
    int now = 0;
    for (char c : s) now = nxt(now, c - 'a');
    cnt[now]++;
    return now;
  }
  void build_fail() {
    queue <int> q;
    for (int i = 0; i < 26; ++i) if (ch[0][i]) {
      to[0][i] = ch[0][i];
      q.push(ch[0][i]);
      g[0].push_back(ch[0][i]);
    }
    while (!q.empty()) {
      int v = q.front(); q.pop();
      for (int j = 0; j < 26; ++j) {
        to[v][j] = ch[v][j] ? ch[v][j] : to[fail[v]][j
            ];
      }
      for (int i = 0; i < 26; ++i) if (ch[v][i]) {
        int u = ch[v][i], k = fail[v];
        while (k && !ch[k][i]) k = fail[k];
        if (ch[k][i]) k = ch[k][i];
        fail[u] = k;
        cnt[u] += cnt[k], g[k].push_back(u);
        q.push(u);
      }
    }
  }
  int match(string &s) {
    int now = 0, ans = 0;
    for (char c : s) {
      now = to[now][c - 'a'];
      ans += cnt[now];
    }
    return ans;
  }
};
```

5.2 Lyndon Factorization [a9eeb0]

```
// partition s = w[0] + w[1] + ... + w[k-1],
// w[0] >= w[1] >= ... >= w[k-1]
// each w[i] strictly smaller than all its suffix
vector <string> duval(const string &s) {
  vector <string> ans;
  for (int n = (int)s.size(), i = 0, j, k; i < n; ) {
    for (j = i + 1, k = i; j < n && s[k] <= s[j]; j++)
      k = (s[k] < s[j] ? i : k + 1);
    for (; i <= k; i += j - k)
      ans.pb(s.substr(i, j - k)); // s.substr(l, len)
  }
  return ans;
}
```

5.3 KMP [5ac553]

```
vector <int> build_fail(string &s) {
  vector <int> f(s.length() + 1, 0);
  int k = 0;
  for (int i = 1; i < s.length(); ++i) {
    while (k && s[k] != s[i])
      k = f[k];
    if (s[k] == s[i])
      k++;
    f[i + 1] = k;
  }
  return f;
}
int match(string &s, string &t) {
  vector <int> f = build_fail(t);
  int k = 0, ans = 0;
  for (int i = 0; i < s.length(); ++i) {
    while (k && s[i] != t[k])
      k = f[k];
    if (s[i] == t[k])
      k++;
    if (k == t.length())
      ans++, k = f[k];
  }
  return ans;
}
```

5.4 Manacher [643e55]

```
int z[MAXN]; // 0-base
/* center i: radius z[i * 2 + 1] / 2
   center i, i + 1: radius z[i * 2 + 2] / 2
   both aba, abba have radius 2 */
void Manacher(string tmp) {
  string s = "%";
  int l = 0, r = 0;
  for (char c : tmp) s.pb(c), s.pb('%');
  for (int i = 0; i < s.size(); ++i) {
    z[i] = r > i ? min(z[2 * l - i], r - i) : 1;
    while (i - z[i] >= 0 && i + z[i] < s.size()
           && s[i + z[i]] == s[i - z[i]]) ++z[i];
    if (z[i] + i > r) r = z[i] + i, l = i;
  }
}
```

5.5 Minimum Rotate [acab8e]

```
string mcp(string s) {
  int n = s.size(), i = 0, j = 1;
  s += s;
  while (i < n && j < n) {
    int k = 0;
    while (k < n && s[i + k] == s[j + k]) k++;
    if (s[i + k] <= s[j + k]) j += k + 1;
    else i += k + 1;
    if (i == j) j++;
  }
  int ans = (i < n ? i : j);
  return s.substr(ans, n);
}
```

5.6 Palindrome Tree [8a071b]

```
struct PalindromicTree {
  struct node {
    int nxt[26], fail, len; // num = depth of fail link
    int cnt, num; // cnt = occur, num = #pal_suffix of
        this node
    node(int l = 0) : nxt{},fail(0),len(l),cnt(0),num
        (0) {}
  };
  vector<node> st; vector<int> s; int last, n;
  void init() {
    st.clear(); s.clear(); last = 1; n = 0;
    st.pb(0); st.pb(-1);
    st[0].fail = 1; s.pb(-1);
  }
  int getFail(int x) {
    while (s[n - st[x].len - 1] != s[n]) x = st[x].fail
        ;
    return x;
  }
  void add(int c) {
    s.pb(c -= 'a'); ++n;
    int cur = getFail(last);
    if (!st[cur].nxt[c]) {
      int now = SZ(st);
      st.pb(st[cur].len + 2);
      st[now].fail = st[getFail(st[cur].fail)].nxt[c];
      st[cur].nxt[c] = now;
      st[now].num = st[st[now].fail].num + 1;
    }
    last = st[cur].nxt[c]; ++st[last].cnt;
```

```
  }
  void dpcnt() {
    for(int i = SZ(st) - 1; i >= 0; i--){
      auto nd = st[i];
      st[nd.fail].cnt += nd.cnt;
    }
  }
  int size() { return (int)st.size() - 2; }
};
```

5.7 Palindrome Partition [c85c05]

```
// in PAM
/* node */ int dif = 0, slink = 0, g = 0;
vector<int> dp = {0};
// add
if (!st[cur].nxt[c]) {
  // ...
  st[now].dif = st[now].len - st[st[now].fail].len;
  if (st[now].dif == st[st[now].fail].dif)
    st[now].slink = st[st[now].fail].slink;
  else st[now].slink = st[now].fail;
}
dp.pb(0);
for (int x = last; x > 1; x = st[x].slink) {
  st[x].g = dp[n - st[st[x].slink].len - st[x].dif];
  if (st[x].dif == st[st[x].fail].dif)
    st[x].g = min(st[x].g, st[st[x].fail].g);
  dp[n] = min(dp[n], st[x].g + 1);
}
```

5.8 Repetition [f3da14]

```
int to_left[N], to_right[N];
vector <array <int, 3>> rep; // l, r, len.
// substr( [l, r], len * 2) are tandem
void findRep(string &s, int l, int r) {
  if (r - l == 1) return;
  int m = l + r >> 1;
  findRep(s, l, m), findRep(s, m, r);
  string sl = s.substr(l, m - l);
  string sr = s.substr(m, r - m);
  vector <int> Z = buildZ(sr + "#" + sl);
  for (int i = l; i < m; ++i)
    to_right[i] = Z[r - m + 1 + i - l];
  reverse(all(sl));
  Z = buildZ(sl);
  for (int i = l; i < m; ++i)
    to_left[i] = Z[m - i - 1];
  reverse(all(sl));
  for (int i = l; i + 1 < m; ++i) {
    int k1 = to_left[i], k2 = to_right[i + 1];
    int len = m - i - 1;
    if (k1 < 1 || k2 < 1 || len < 2) continue;
    int tl = max(1, len - k2), tr = min(len - 1, k1);
    if (tl <= tr) rep.pb({i + 1 - tr, i + 1 - tl,len});
  }
  Z = buildZ(sr);
  for (int i = m; i < r; ++i) to_right[i] = Z[i - m];
  reverse(all(sl)), reverse(all(sr));
  Z = buildZ(sl + "#" + sr);
  for (int i = m; i < r; ++i)
    to_left[i] = Z[m - l + 1 + r - i - 1];
  reverse(all(sl)), reverse(all(sr));
  for (int i = m; i + 1 < r; ++i) {
    int k1 = to_left[i], k2 = to_right[i + 1];
    int len = i - m + 1;
    if (k1 < 1 || k2 < 1 || len < 2) continue;
    int tl = max(len - k2, 1), tr = min(len - 1, k1);
    if (tl <= tr)
      rep.pb({i + 1 - len - tr, i + 1 - len - tl,len});
  }
  Z = buildZ(sr + "#" + sl);
  for (int i = l; i < m; ++i)
    if (Z[r - m + 1 + i - l] >= m - i)
      rep.pb({i, i, m - i});
}
```

5.9 Suffix Array [60547e]

```
#define FOR(i, a, b) for(int i = a; i <= b; i++)
#define ROF(i, a, b) for(int i = a; i >= b; i--)
int sa[N], tmp[2][N], c[N], rk[N], lcp[N];
void buildSA(string s) {
  int *x = tmp[0], *y = tmp[1], m = 256, n = (int)s.
      size();
  FOR(i, 0, m - 1) c[i] = 0;
  FOR(i, 0, n - 1) c[x[i] = s[i]]++;
  FOR(i, 1, m - 1) c[i] += c[i - 1];
  ROF(i, n - 1, 0) sa[--c[x[i]]] = i;
  for (int k = 1; k < n; k <<= 1) {
    FOR(i, 0, m - 1) c[i] = 0;
    FOR(i, 0, n - 1) c[x[i]]++;
    FOR(i, 1, m - 1) c[i] += c[i - 1];
    int p = 0;
    FOR(i, n - k, n - 1) y[p++] = i;
    FOR(i, 0, n - 1) if(sa[i] >= k) y[p++] = sa[i] - k;
    ROF(i, n - 1, 0) sa[--c[x[y[i]]]] = y[i];
    y[sa[0]] = p = 0;
    FOR(i, 1, n - 1) {
      int a = sa[i], b = sa[i - 1];
      if (!(x[a] == x[b] && a + k < n && b + k < n && x
          [a + k] == x[b + k])) p++;
      y[sa[i]] = p;
    }
    if (n == p + 1) break;
    swap(x, y), m = p + 1;
  }
}
void buildLCP(string s) {
  // lcp[i] = LCP(sa[i - 1], sa[i])
  // lcp(i, j) = min(lcp[rk[i] + 1], lcp[rk[i] + 2],
      ..., lcp[rk[j]])
  int n = (int)s.size(), val = 0;
  FOR(i, 0, n - 1) rk[sa[i]] = i;
  FOR(i, 0, n - 1) {
    if (!rk[i]) lcp[rk[i]] = 0;
    else {
      if (val) val--;
      int p = sa[rk[i] - 1];
      while (val + i < n && val + p < n && s[val + i]
          == s[val + p]) val++;
      lcp[rk[i]] = val;
    }
  }
}
```

5.10 SAIS (C++20) [6f26bc]

```
auto sais(const auto &s) {
  const int n = SZ(s), z = ranges::max(s) + 1;
  if (n == 1) return vector{0};
  vector<int> c(z); for (int x : s) ++c[x];
  partial_sum(ALL(c), begin(c));
  vector<int> sa(n); auto I = views::iota(0, n);
  vector<bool> t(n, true);
  for (int i = n - 2; i >= 0; --i)
    t[i] = (s[i] == s[i + 1] ? t[i + 1] : s[i] < s[i +
        1]);
  auto is_lms = views::filter([&t](int x) {
    return x && t[x] && !t[x - 1];
  });
  auto induce = [&] {
    for (auto x = c; int y : sa)
      if (y--) if (!t[y]) sa[x[s[y] - 1]++] = y;
    for (auto x = c; int y : sa | views::reverse)
      if (y--) if (t[y]) sa[--x[s[y]]] = y;
  };
  vector<int> lms, q(n); lms.reserve(n);
  for (auto x = c; int i : I | is_lms)
    q[i] = SZ(lms), lms.pb(sa[--x[s[i]]] = i);
  induce(); vector<int> ns(SZ(lms));
  for (int j = -1, nz = 0; int i : sa | is_lms) {
    if (j >= 0) {
      int len = min({n - i, n - j, lms[q[i] + 1] - i});
      ns[q[i]] = nz += lexicographical_compare(
          begin(s) + j, begin(s) + j + len,
          begin(s) + i, begin(s) + i + len);
    }
    j = i;
  }
  fill(ALL(sa), 0); auto nsa = sais(ns);
  for (auto x = c; int y : nsa | views::reverse)
    y = lms[y], sa[--x[s[y]]] = y;
```

```
  return induce(), sa;
}
// sa[i]: sa[i]-th suffix is the i-th lexicographically
     smallest suffix.
// hi[i]: LCP of suffix sa[i] and suffix sa[i - 1].
struct Suffix {
  int n; vector<int> sa, hi, ra;
  Suffix(const auto &_s, int _n) : n(_n), hi(n), ra(n)
      {
    vector<int> s(n + 1); // s[n] = 0;
    copy_n(_s, n, begin(s)); // _s shouldn't contain 0
    sa = sais(s); sa.erase(sa.begin());
    for (int i = 0; i < n; ++i) ra[sa[i]] = i;
    for (int i = 0, h = 0; i < n; ++i) {
      if (!ra[i]) { h = 0; continue; }
      for (int j = sa[ra[i] - 1]; max(i, j) + h < n &&
          s[i + h] == s[j + h];) ++h;
      hi[ra[i]] = h ? h-- : 0;
    }
  }
};
```

5.11 Suffix Automaton [277d1d]

```
struct SAM {
  int ch[N][26], len[N], link[N], pos[N], cnt[N], sz;
  // node -> strings with the same endpos set
  // length in range [len(link) + 1, len]
  // node's endpos set -> pos in the subtree of node
  // link -> longest suffix with different endpos set
  // len  -> longest suffix
  // pos  -> end position
  // cnt  -> size of endpos set
  SAM () {len[0] = 0, link[0] = -1, pos[0] = 0, cnt[0]
      = 0, sz = 1;}
  void build(string s) {
    int last = 0;
    for (int i = 0; i < s.length(); ++i) {
      char c = s[i];
      int cur = sz++;
      len[cur] = len[last] + 1, pos[cur] = i + 1;
      int p = last;
      while (~p && !ch[p][c - 'a'])
        ch[p][c - 'a'] = cur, p = link[p];
      if (p == -1) link[cur] = 0;
      else {
        int q = ch[p][c - 'a'];
        if (len[p] + 1 == len[q]) {
          link[cur] = q;
        } else {
          int nxt = sz++;
          len[nxt] = len[p] + 1, link[nxt] = link[q];
          pos[nxt] = pos[q];
          for (int j = 0; j < 26; ++j)
            ch[nxt][j] = ch[q][j];
          while (~p && ch[p][c - 'a'] == q)
            ch[p][c - 'a'] = nxt, p = link[p];
          link[q] = link[cur] = nxt;
        }
      }
      cnt[cur]++;
      last = cur;
    }
    vector <int> p(sz);
    iota(all(p), 0);
    sort(all(p),
      [&](int i, int j) {return len[i] > len[j];});
    for (int i = 0; i < sz; ++i)
      cnt[link[p[i]]] += cnt[p[i]];
  }
} sam;
```

5.12 Z Value [a8c33c]

```
vector <int> build(string s) {
  int n = s.length();
  vector <int> Z(n);
  int l = 0, r = 0;
  for (int i = 0; i < n; ++i) {
    Z[i] = max(min(Z[i - l], r - i), 0);
    while (i + Z[i] < s.size() && s[Z[i]] == s[i + Z[i
        ]]) {
      l = i, r = i + Z[i], Z[i]++;
    }
  }
  return Z;
}
```

6 Math

6.1 Characteristic Polynomial [eabd3c]

```
#define rep(x, y, z) for (int x=y; x<z; x++)
using vi = vector<int>; using vvi = vector<vi>;
void Hessenberg(vvi &H, int N) { // 0ddcb8
  for (int i = 0; i < N - 2; ++i) {
    for (int j = i + 1; j < N; ++j) if (H[j][i]) {
      rep(k, i, N) swap(H[i+1][k], H[j][k]);
      rep(k, 0, N) swap(H[k][i+1], H[k][j]);
      break;
    }
    if (!H[i + 1][i]) continue;
    for (int j = i + 2; j < N; ++j) {
      int co = mul(modinv(H[i + 1][i]), H[j][i]);
      rep(k, i, N) subeq(H[j][k], mul(H[i+1][k], co));
      rep(k, 0, N) addeq(H[k][i+1], mul(H[k][j], co));
    }
  }
}
vi CharacteristicPoly(vvi A) { // 5213a6
  int N = (int)A.size(); Hessenberg(A, N);
  vvi P(N + 1, vi(N + 1)); P[0][0] = 1;
  for (int i = 1; i <= N; ++i) {
    rep(j, 0, i+1) P[i][j] = j ? P[i-1][j-1] : 0;
    for (int j = i - 1, val = 1; j >= 0; --j) {
      int co = mul(val, A[j][i - 1]);
      rep(k, 0, j+1) subeq(P[i][k], mul(P[j][k], co));
      if (j) val = mul(val, A[j][j - 1]);
    }
  }
  if (N & 1) for (int &x: P[N]) x = sub(0, x);
  return P[N]; // test: 2021 PTZ Korea K
}
```

6.2 Discrete Logarithm [4c39b3]

```
int BSGS(int s, int x, int y, int m) { // 247dc4
  constexpr int kStep = 32000;
  unordered_map<int, int> p;
  int b = 1;
  for (int i = 0; i < kStep; ++i) {
    p[y] = i;
    y = 1ll * y * x % m;
    b = 1ll * b * x % m;
  }
  for (int i = 0; i < m + 10; i += kStep) {
    s = 1ll * s * b % m;
    if (p.find(s) != p.end()) return i + kStep - p[s];
  }
  return -1; // returns: j where s*(x^j) = y (mod m)
}
int DiscreteLog(int x, int y, int m) { // 579300
  if (m == 1) return 0;
  int s = 1;
  for (int i = 0; i < 100; ++i) {
    if (s == y) return i;
    s = 1ll * s * x % m;
  }
  if (s == y) return 100;
  int p = 100 + BSGS(s, x, y, m);
  if (fpow(x, p, m) != y) return -1;
  return p; //returns: x^p = y (mod m)
}
```

6.3 Extgcd [c51ae9]

```
// ax+ny = 1, ax+ny == ax == 1 (mod n)
void extgcd(ll x, ll y, ll &g, ll &a, ll &b) {
  if (y == 0) g = x, a = 1, b = 0;
  else extgcd(y, x % y, g, b, a), b -= (x / y) * a;
}
```

6.4 Floor Sum [49de67]

```
// sum^{n-1}_0 floor((a * i + b) / m) in log(n + m + a
    + b)
ll floor_sum(ll n, ll m, ll a, ll b) {
  ll ans = 0;
  if (a >= m) ans += (n - 1) * n * (a / m) / 2, a %= m;
  if (b >= m) ans += n * (b / m), b %= m;
  ll y_max = (a * n + b) / m, x_max = (y_max * m - b);
  if (y_max == 0) return ans;
  ans += (n - (x_max + a - 1) / a) * y_max;
  ans += floor_sum(y_max, a, m, (a - x_max % a) % a);
  return ans;
}
```

6.5 Factorial Mod P^k [c324f3]

```
// O(p^k + log^2 n), pk = p^k
ll prod[MAXP];
ll fac_no_p(ll n, ll p, ll pk) {
  prod[0] = 1;
  for (int i = 1; i <= pk; ++i)
    if (i % p) prod[i] = prod[i - 1] * i % pk;
    else prod[i] = prod[i - 1];
  ll rt = 1;
  for (; n; n /= p) {
    rt = rt * mpow(prod[pk], n / pk, pk) % pk;
    rt = rt * prod[n % pk] % pk;
  }
  return rt;
} // (n! without factor p) % p^k
```

6.6 Gaussian Elimination [0465e4]

```
using vi = vector<int>; // be careful if A.empty()
using vvi = vector<vi>; // ensure that 0 <= x < mod
pair<vi, vvi> gauss(vvi A, vi b) { // solve Ax=b
  const int N = (int)A.size(), M = (int)A[0].size();
  vector<int> depv, free(M, true); int rk = 0;
  for (int i = 0; i < M; i++) {
    int p = -1;
    for (int j = rk; j < N; j++)
      if (p == -1 || abs(A[j][i]) > abs(A[p][i]))
        p = j;
    if (p == -1 || A[p][i] == 0) continue;
    swap(A[p], A[rk]); swap(b[p], b[rk]);
    const int inv = modinv(A[rk][i]);
    for (int &x : A[rk]) x = mul(x, inv);
    b[rk] = mul(b[rk], inv);
    for (int j = 0; j < N; j++) if (j != rk) {
      int z = A[j][i];
      for (int k = 0; k < M; k++)
        A[j][k] = sub(A[j][k], mul(z, A[rk][k]));
      b[j] = sub(b[j], mul(z, b[rk]));
    }
    depv.push_back(i); free[i] = false; ++rk;
  }
  for (int i = rk; i < N; i++)
    if (b[i] != 0) return {{}, {}}; // not consistent
  vi x(M); vvi h;
  for (int i = 0; i < rk; i++) x[depv[i]] = b[i];
  for (int i = 0; i < M; i++) if (free[i]) {
    h.emplace_back(M); h.back()[i] = 1;
    for (int j = 0; j < rk; j++)
      h.back()[depv[j]] = sub(0, A[j][i]);
  }
  return {x, h}; // solution = x + span(h[i])
}
```

6.7 Linear Function Mod Min [93cc4b]

```
ll topos(ll x, ll m) {x %= m; if (x < 0) x += m; return
    x;}
//min value of ax + b (mod m) for x \in [0, n - 1]. O(
    log m)
ll min_rem(ll n, ll m, ll a, ll b) { // 578b62
  a = topos(a, m), b = topos(b, m);
  for (ll g = __gcd(a, m); g > 1;) return g * min_rem(n
      , m / g, a / g, b / g) + (b % g);
  for (ll nn, nm, na, nb; a; n = nn, m = nm, a = na, b
      = nb) {
    if (a <= m - a) {
      nn = (a * (n - 1) + b) / m;
      if (!nn) break;
      nn += (b < a);
      nm = a, na = topos(-m, a);
      nb = b < a ? b : topos(b - m, a);
    } else {
      ll lst = b - (n - 1) * (m - a);
      if (lst >= 0) {b = lst; break;}
      nn = -(lst / m) + (lst % m < -a) + 1;
      nm = m - a, na = m % (m - a), nb = b % (m - a);
    }
  }
  return b;
}
//min value of ax + b (mod m) for x \in [0, n - 1],
    also return min x to get the value. O(log m)
//{value, x}
pair<ll, ll> min_rem_pos(ll n, ll m, ll a, ll b) { //
    fd6550
  a = topos(a, m), b = topos(b, m);
  ll mn = min_rem(n, m, a, b), g, ia, ib;
  extgcd(a, m, g, ia, ib);
  //ax = (mn - b) (mod m)
  ll x = (ia + m) * ((mn - b + m) / g) % (m / g);
  return {mn, x};
}
```

6.8 MillerRabin PollardRho [9d0778]

```
// n < 4,759,123,141        3 : 2, 7, 61
// n < 1,122,004,669,633    4 : 2, 13, 23, 1662803
// n < 3,474,749,660,383    6 : primes <= 13
// n < 2^64                 7 :
// 2, 325, 9375, 28178, 450775, 9780504, 1795265022
ll mul(ll a, ll b, ll n){  return (__int128)a * b % n;
    }
vector<ll> chk = {2, 325, 9375, 28178, 450775, 9780504,
     1795265022};
ll Pow(ll a, ll b, ll n) {ll res = 1; for (; b; b >>=
    1, a = mul(a, a, n)) if (b & 1) res = mul(res, a, n
    ); return res;}
bool check(ll a, ll d, int s, ll n) { // 874cd4
  a = Pow(a, d, n);
  if (a <= 1) return 1;
  for (int i = 0; i < s; ++i, a = mul(a, a, n)) {
    if (a == 1) return 0;
    if (a == n - 1) return 1;
  }
  return 0;
}
bool IsPrime(ll n) { // b8bfee
  if (n < 2) return 0;
  if (n % 2 == 0) return n == 2;
  ll d = n - 1, s = 0;
  while (d % 2 == 0) d >>= 1, ++s;
  for (ll i : chk) if (!check(i, d, s, n)) return 0;
  return 1;
}
const vector<ll> small = {2, 3, 5, 7, 11, 13, 17, 19};
ll FindFactor(ll n) { // 697b88
  if (IsPrime(n)) return 1;
  for (ll p : small) if (n % p == 0) return p;
  ll x, y = 2, d, t = 1;
  auto f = [&](ll a) {return (mul(a, a, n) + t) % n;};
  for (int l = 2; ; l <<= 1) {
    x = y;
    int m = min(l, 32);
    for (int i = 0; i < l; i += m) {
      d = 1;
      for (int j = 0; j < m; ++j) {
        y = f(y), d = mul(d, abs(x - y), n);
      }
      ll g = __gcd(d, n);
      if (g == n) {
        l = 1, y = 2, ++t;
        break;
      }
      if (g != 1) return g;
    }
  }
}
map<ll, int> PollardRho(ll n) { // 1cf996
  map<ll, int> res;
  if (n == 1) return res;
```

```
  if (IsPrime(n)) return ++res[n], res;
  ll d = FindFactor(n);
  res = PollardRho(n / d);
  auto res2 = PollardRho(d);
  for (auto [x, y] : res2) res[x] += y;
  return res;
}
```

6.9 Quadratic Residue [ac688c]

```
int Jacobi(int a, int m) { // 00fa11
  int s = 1;
  for (; m > 1; ) {
    a %= m;
    if (a == 0) return 0;
    const int r = __builtin_ctz(a);
    if ((r & 1) && ((m + 2) & 4)) s = -s;
    a >>= r;
    if (a & m & 2) s = -s;
    swap(a, m);
  }
  return s;
}
int QuadraticResidue(int a, int p) { // 1a2d08
  if (p == 2) return a & 1;
  const int jc = Jacobi(a, p);
  if (jc == 0) return 0;
  if (jc == -1) return -1;
  int b, d;
  for (; ; ) {
    b = rand() % p;
    d = (1ll * b * b + p - a) % p;
    if (Jacobi(d, p) == -1) break;
  }
  int f0 = b, f1 = 1, g0 = 1, g1 = 0, tmp;
  for (int e = (1ll + p) >> 1; e; e >>= 1) {
    if (e & 1) {
      tmp = (1ll * g0 * f0 + 1ll * d * (1ll * g1 * f1 %
           p)) % p;
      g1 = (1ll * g0 * f1 + 1ll * g1 * f0) % p;
      g0 = tmp;
    }
    tmp = (1ll * f0 * f0 + 1ll * d * (1ll * f1 * f1 % p
         )) % p;
    f1 = (2ll * f0 * f1) % p;
    f0 = tmp;
  }
  return g0;
}
```

6.10 Simplex [ece29e]

```
struct Simplex { // 0-based
  using T = long double;
  static const int N = 410, M = 30010;
  const T eps = 1e-7;
  int n, m;
  int Left[M], Down[N];
  // Ax <= b, max c^T x. result : v, xi = sol[i]. 1-
      based
  T a[M][N], b[M], c[N], v, sol[N];
  bool eq(T a, T b) {return fabs(a - b) < eps;}
  bool ls(T a, T b) {return a < b && !eq(a, b);}
  void init(int _n, int _m) { // a752a0
    n = _n, m = _m, v = 0;
    for (int i = 0; i < m; ++i) for (int j = 0; j < n;
        ++j) a[i][j] = 0;
    for (int i = 0; i < m; ++i) b[i] = 0;
    for (int i = 0; i < n; ++i) c[i] = sol[i] = 0;
  }
  void pivot(int x, int y) { // 92faa6
    swap(Left[x], Down[y]);
    T k = a[x][y]; a[x][y] = 1;
    vector <int> nz;
    for (int i = 0; i < n; ++i) {
      a[x][i] /= k;
      if (!eq(a[x][i], 0)) nz.push_back(i);
    }
    b[x] /= k;
    for (int i = 0; i < m; ++i) {
      if (i == x || eq(a[i][y], 0)) continue;
      k = a[i][y], a[i][y] = 0;
      b[i] -= k * b[x];
      for (int j : nz) a[i][j] -= k * a[x][j];
    }
    if (eq(c[y], 0)) return;
    k = c[y], c[y] = 0, v += k * b[x];
    for (int i : nz) c[i] -= k * a[x][i];
  }
  // 0: found solution, 1: no feasible solution, 2:
      unbounded
  int solve() { // 9f4550
    for (int i = 0; i < n; ++i) Down[i] = i;
    for (int i = 0; i < m; ++i) Left[i] = n + i;
    while (1) {
      int x = -1, y = -1;
      for (int i = 0; i < m; ++i) if (ls(b[i], 0) && (x
          == -1 || b[i] < b[x])) x = i;
      if (x == -1) break;
      for (int i = 0; i < n; ++i) if (ls(a[x][i], 0) &&
          (y == -1 || a[x][i] < a[x][y])) y = i;
      if (y == -1) return 1;
      pivot(x, y);
    }
    while (1) {
      int x = -1, y = -1;
      for (int i = 0; i < n; ++i) if (ls(0, c[i]) && (y
          == -1 || c[i] > c[y])) y = i;
      if (y == -1) break;
      for (int i = 0; i < m; ++i) if (ls(0, a[i][y]) &&
          (x == -1 || b[i] / a[i][y] < b[x] / a[x][y])
         ) x = i;
      if (x == -1) return 2;
      pivot(x, y);
    }
    for (int i = 0; i < m; ++i) if (Left[i] < n) sol[
        Left[i]] = b[i];
    return 0;
  }
};
```

6.11 Linear Programming Construction

Standard form: maximize $\mathbf{c}^T\mathbf{x}$ subject to $A\mathbf{x} \le \mathbf{b}$ and $\mathbf{x} \ge 0$.
Dual LP: minimize $\mathbf{b}^T\mathbf{y}$ subject to $A^T\mathbf{y} \ge \mathbf{c}$ and $\mathbf{y} \ge 0$.
$\bar{\mathbf{x}}$ and $\bar{\mathbf{y}}$ are optimal if and only if for all $i \in [1, n]$, either $\bar{x}_i = 0$ or $\sum_{j=1}^{m} A_{ji}\bar{y}_j = c_i$ holds and for all $i \in [1, m]$ either $\bar{y}_i = 0$ or $\sum_{j=1}^{n} A_{ij}\bar{x}_j = b_j$ holds.

1. In case of minimization, let $c'_i = -c_i$
2. $\sum_{1\le i\le n} A_{ji}x_i \ge b_j \to \sum_{1\le i\le n} -A_{ji}x_i \le -b_j$
3. $\sum_{1\le i\le n} A_{ji}x_i = b_j$
   - $\sum_{1\le i\le n} A_{ji}x_i \le b_j$
   - $\sum_{1\le i\le n} A_{ji}x_i \ge b_j$
4. If x_i has no lower bound, replace x_i with $x_i - x'_i$

6.12 Estimation

- Number of paritions of n:

n	2	3	4	5	6	7	8	9	20	30	40	50	100
$p(n)$	2	3	5	7	11	15	22	30	627	5604	4e4	2e5	2e8

- Maximum number of divisors:

n	100	1e3	1e6	1e9	1e12	1e15	1e18
$d(i)$	12	32	240	1344	6720	26880	103680
arg	60	840	720720	735134400	963761198400	866421317361600	89761248478661

-

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\binom{2n}{n}$	2	6	20	70	252	924	3432	12870	48620	184756	7e5	2e6	1e7	4e7	1.5e8

- Number of ways to partition a set of n labeled elements:

n	2	3	4	5	6	7	8	9	10	11	12	13
B_n	2	5	15	52	203	877	4140	21147	115975	7e5	4e6	3e7

6.13 Theorem

- Kirchhoff's Theorem

  Denote L be a $n \times n$ matrix as the Laplacian matrix of graph G, where $L_{ii} = d(i)$, $L_{ij} = -c$ where c is the number of edge (i, j) in G.

  - The number of undirected spanning in G is $|\det(\tilde{L}_{11})|$.
  - The number of directed spanning tree rooted at r in G is $|\det(\tilde{L}_{rr})|$.

- Tutte's Matrix

  Let D be a $n \times n$ matrix, where $d_{ij} = x_{ij}$ (x_{ij} is chosen uniformly at random) if $i < j$ and $(i, j) \in E$, otherwise $d_{ij} = -d_{ji}$. $\frac{rank(D)}{2}$ is the maximum matching on G.

- Cayley's Formula
  - Given a degree sequence $d_1, d_2, \ldots, d_n$ for each *labeled* vertices, there are
    $$\frac{(n-2)!}{(d_1-1)!(d_2-1)!\cdots(d_n-1)!}$$
    spanning trees.
  - Let $T_{n,k}$ be the number of *labeled* forests on n vertices with k components, such that vertex $1, 2, \ldots, k$ belong to different components. Then $T_{n,k} = kn^{n-k-1}$.
- Erdős–Gallai Theorem

  A sequence of non-negative integers $d_1 \geq d_2 \geq \ldots \geq d_n$ can be represented as the degree sequence of a finite simple graph on n vertices if and only if $d_1 + d_2 + \ldots + d_n$ is even and
  $$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$
  holds for all $1 \leq k \leq n$.
- Burnside's Lemma

  Let X be a set and G be a group that acts on X. For $g \in G$, denote by X^g the elements fixed by g:
  $$X^g = \{x \in X \mid gx \in X\}$$
  Then
  $$|X/G| = \frac{1}{|G|}\sum_{g \in G} |X^g|.$$
- Gale–Ryser theorem

  A pair of sequences of nonnegative integers $a_1 \geq \cdots \geq a_n$ and $b_1, \ldots, b_n$ is bigraphic if and only if $\sum_{i=1}^{n} a_i = \sum_{i=1}^{n} b_i$ and $\sum_{i=1}^{k} a_i \leq \sum_{i=1}^{n} \min(b_i, k)$ holds for every $1 \leq k \leq n$.
- Fulkerson–Chen–Anstee theorem

  A sequence $(a_1, b_1), \ldots, (a_n, b_n)$ of nonnegative integer pairs with $a_1 \geq \cdots \geq a_n$ is digraphic if and only if $\sum_{i=1}^{n} a_i = \sum_{i=1}^{n} b_i$ and $\sum_{i=1}^{k} a_i \leq \sum_{i=1}^{k} \min(b_i, k-1) + \sum_{i=k+1}^{n} \min(b_i, k)$ holds for every $1 \leq k \leq n$.
- Möbius inversion formula
  - $f(n) = \sum_{d \mid n} g(d) \Leftrightarrow g(n) = \sum_{d \mid n} \mu(d) f(\frac{n}{d})$
  - $f(n) = \sum_{n \mid d} g(d) \Leftrightarrow g(n) = \sum_{n \mid d} \mu(\frac{d}{n}) f(d)$
- Spherical cap
  - A portion of a sphere cut off by a plane.
  - r: sphere radius, a: radius of the base of the cap, h: height of the cap, θ: $\arcsin(a/r)$.
  - Volume $= \pi h^2(3r-h)/3 = \pi h(3a^2+h^2)/6 = \pi r^3(2+\cos\theta)(1-\cos\theta)^2/3$.
  - Area $= 2\pi rh = \pi(a^2+h^2) = 2\pi r^2(1-\cos\theta)$.
- Chinese Remainder Theorem
  - $x \equiv a_i \pmod{m_i}$
  - $M = \prod m_i, M_i = M/m_i$
  - $t_i M_i \equiv 1 \pmod{m_i}$
  - $x = \sum a_i t_i M_i \pmod{M}$
- Green Hackenbush: Run BCC(bridge), each BCC has grundy value $g_v = |E| \bmod 2$ (the grundy value remains unchanged if we identify a circuit as a node). Run DFS with root *ground* and $g_v \leftarrow g_v \oplus (g_h + 1)$ for vertex v and its child h.

6.14 General Purpose Numbers

- Bernoulli numbers

  $B_0 = 1, B_1^{\pm} = \pm\frac{1}{2}, B_2 = \frac{1}{6}, B_3 = 0$

  $\sum_{j=0}^{m}\binom{m+1}{j}B_j = 0$, EGF is $B(x) = \frac{x}{e^x-1} = \sum_{n=0}^{\infty} B_n \frac{x^n}{n!}$.

  $$S_m(n) = \sum_{k=1}^{n} k^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k^+ n^{m+1-k}$$
- Stirling numbers of the second kind Partitions of n distinct elements into exactly k groups.

  $S(n,k) = S(n-1,k-1) + kS(n-1,k), S(n,1) = S(n,n) = 1$

  $S(n,k) = \frac{1}{k!}\sum_{i=0}^{k}(-1)^{k-i}\binom{k}{i}i^n$

  $x^n = \sum_{i=0}^{n} S(n,i)(x)_i$
- Pentagonal number theorem
  $$\prod_{n=1}^{\infty}(1-x^n) = 1 + \sum_{k=1}^{\infty}(-1)^k\left(x^{k(3k+1)/2} + x^{k(3k-1)/2}\right)$$
- Catalan numbers

  $C_n^{(k)} = \frac{1}{(k-1)n+1}\binom{kn}{n}$

  $C^{(k)}(x) = 1 + x[C^{(k)}(x)]^k$
- Eulerian numbers

  Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

  $E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$

  $E(n,0) = E(n,n-1) = 1$

  $E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$

6.15 Integral

$\frac{d}{dx}$	$\sec^2 x$	$-\csc^2 x$	$\tan x \sec x$	$-\cot x \csc x$
$f(x)$	$\tan x$	$\cot x$	$\sec x$	$\csc x$
$\int$	$\ln\lvert\sec x\rvert$	$-\ln\lvert\csc x\rvert$	$\ln\lvert\tan x + \sec x\rvert$	$-\ln\lvert\cot x + \csc x\rvert$
$\frac{d}{dx}$	$\frac{1}{\sqrt{1-x^2}}$	$\frac{-1}{\sqrt{1-x^2}}$	$\frac{1}{x^2+1}$	$\sec^3 x$
$f(x)$	$\operatorname{asin} x$	$\operatorname{acos} x$	$\operatorname{atan} x$	$\frac{1}{2}(\sec x \tan x + \ln\lvert\sec x + \tan x\rvert)$

- $\int_0^{\frac{\pi}{2}} \sin^{2k} dx = \frac{\pi}{2}\cdot\frac{1\cdot 3\cdots(2k-1)}{2\cdot 4\cdots(2k)}$ $\quad\int_0^{\frac{\pi}{2}} \sin^{2k+1} dx = \frac{2\cdot 4\cdots(2k)}{3\cdot 5\cdots(2k+1)}$
- $\iint f(x,y)\,dx\,dy = \iint f(g(u,v),h(u,v))\begin{vmatrix}\partial x/\partial u & \partial x/\partial v\\ \partial y/\partial u & \partial y/\partial v\end{vmatrix} du\,dv$
- polar: $\iint r \cdot f(r\cos\theta, r\sin\theta)\,dr\,d\theta$

  spherical: $\iiint \rho^2 \sin\phi \cdot f(\rho\sin\phi\cos\theta, \rho\sin\phi\sin\theta, \rho\cos\theta)\,d\phi\,d\theta\,d\rho$

6.16 Floor Sum

- $m = \lfloor\frac{an+b}{c}\rfloor$
- Time complexity: $O(\log n)$

$$f(a,b,c,n) = \sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor$$
$$= \begin{cases} \lfloor\frac{a}{c}\rfloor\cdot\frac{n(n+1)}{2} + \lfloor\frac{b}{c}\rfloor\cdot(n+1) & \\ +f(a \bmod c, b \bmod c, c, n), & a \geq c \lor b \geq c \\ 0, & n < 0 \lor a = 0 \\ nm - f(c, c-b-1, a, m-1), & \text{otherwise} \end{cases}$$

$$g(a,b,c,n) = \sum_{i=0}^{n} i\lfloor\frac{ai+b}{c}\rfloor$$
$$= \begin{cases} \lfloor\frac{a}{c}\rfloor\cdot\frac{n(n+1)(2n+1)}{6} + \lfloor\frac{b}{c}\rfloor\cdot\frac{n(n+1)}{2} & \\ +g(a \bmod c, b \bmod c, c, n), & a \geq c \lor b \geq c \\ 0, & n < 0 \lor a = 0 \\ \frac{1}{2}\cdot(n(n+1)m - f(c, c-b-1, a, m-1) & \\ -h(c, c-b-1, a, m-1)), & \text{otherwise} \end{cases}$$

$$h(a,b,c,n) = \sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor^2$$
$$= \begin{cases} \lfloor\frac{a}{c}\rfloor^2\cdot\frac{n(n+1)(2n+1)}{6} + \lfloor\frac{b}{c}\rfloor^2\cdot(n+1) & \\ +\lfloor\frac{a}{c}\rfloor\cdot\lfloor\frac{b}{c}\rfloor\cdot n(n+1) & \\ +h(a \bmod c, b \bmod c, c, n) & \\ +2\lfloor\frac{a}{c}\rfloor\cdot g(a \bmod c, b \bmod c, c, n) & \\ +2\lfloor\frac{b}{c}\rfloor\cdot f(a \bmod c, b \bmod c, c, n), & a \geq c \lor b \geq c \\ 0, & n < 0 \lor a = 0 \\ nm(m+1) - 2g(c, c-b-1, a, m-1) & \\ -2f(c, c-b-1, a, m-1) - f(a,b,c,n), & \text{otherwise} \end{cases}$$

7 Polynomial

7.1 FFT [43cc28]

```cpp
const double pi=acos(-1);
typedef complex<double> cp;
const int N=(1<<17);
struct FFT
{
  // n has to be same as a.size()
  int n,rev[N];
  cp omega[N],iomega[N];
```

```
  void init(int _n) {
    n=_n;
    for(int i=0;i<n;i++) {
      omega[i]=cp(cos(2*pi/n*i),sin(2*pi/n*i));
      iomega[i]=conj(omega[i]);
    }
    int k=log2(n);
    for(int i=0;i<n;i++) {
      rev[i]=0;
      for(int j=0;j<k;j++) if(i&(1<<j))
        rev[i]|=(1<<(k-j-1));
    }
  }
  void tran(vector<cp> &a,cp* xomega)
  {
    for(int i=0;i<n;i++) if(i<rev[i])
      swap(a[i],a[rev[i]]);
    for(int len=2;len<=n;len<<=1) {
      int mid=len>>1,r=n/len;
      for(int j=0;j<n;j+=len) {
        for(int i=0;i<mid;i++) {
          cp t=xomega[r*i]*a[j+mid+i];
          a[j+mid+i]=a[j+i]-t;
          a[j+i]+=t;
        }
      }
    }
  }
  void fft(vector<cp> &a) {tran(a,omega);}
  void ifft(vector<cp> &a) {
      tran(a,iomega);
      for(int i=0;i<n;i++) a[i]/=n;
  }
};
```

7.2 NTT [f68103]

```
//needs fpow
//needs inv

//(2^16)+1, 65537, 3
//7*17*(2^23)+1, 998244353, 3
//1255*(2^20)+1, 1315962881, 3
//51*(2^25)+1, 1711276033, 29
template<int MAXN, ll P, ll RT> //MAXN must be 2^k
struct NTT {
  ll w[MAXN];
  ll mpow(ll a, ll n);
  ll minv(ll a) { return mpow(a, P - 2); }
  NTT() {
    ll dw = mpow(RT, (P - 1) / MAXN);
    w[0] = 1;
    for (int i = 1; i < MAXN; ++i) w[i] = w[i - 1] * dw
        % P;
  }
  void bitrev(ll *a, int n) {
    int i = 0;
    for (int j = 1; j < n - 1; ++j) {
      for (int k = n >> 1; (i ^= k) < k; k >>= 1);
      if (j < i) swap(a[i], a[j]);
    }
  }
  void operator()(ll *a, int n, bool inv = false) { //0
      <= a[i] < P
    bitrev(a, n);
    for (int L = 2; L <= n; L <<= 1) {
      int dx = MAXN / L, dl = L >> 1;
      for (int i = 0; i < n; i += L) {
        for (int j = i, x = 0; j < i + dl; ++j, x += dx
            ) {
          ll tmp = a[j + dl] * w[x] % P;
          if ((a[j + dl] = a[j] - tmp) < 0) a[j + dl]
              += P;
          if ((a[j] += tmp) >= P) a[j] -= P;
        }
      }
    }
    if (inv) {
      reverse(a + 1, a + n);
      ll invn = minv(n);
      for (int i = 0; i < n; ++i) a[i] = a[i] * invn %
          P;
      }
    }
};
```

7.3 FWT [d4ce4e]

```
/* (op, invop): (oyx: op from y to x)
or: (1, 0, 1, 1), (1, 0, -1, 1)
and: (1, 1, 0, 1), (1, -1, 0, 1)
xor: (1, 1, 1, -1), (1/2, 1/2, 1/2, -1/2) */
void fwt(vector<int>& a, int n, int oxx, int oyx, int
    oxy, int oyy){ // 2^20, 0.5s
  for(int L=2;L<=n;L<<=1)
    for(int i=0;i<n;i+=L)
      for(int j=i;j<i+(L>>1);j++){
#define x a[j]
#define y a[j+(L>>1)]
        tie(x, y) = pii(mul(x,oxx)+mul(y,oyx), mul(x,
            oxy)+mul(y,oyy));
        add(x, 0); add(y, 0);
#undef x
#undef y
      }
}

vector<int> subset_conv(vector<int> a, vector<int> b,
    int L){
  int n = 1<<L; // 2^20, 5.5s, using c-sytle arrays ->
      2.1s
  vector<vector<int> > f(L+1, vector<int>(n)), g=f, h=f
      ;
  vector<int> ct(n), c(n);
  for(int i=1;i<n;i++) ct[i] = ct[i&(i-1)] + 1;
  for(int i=0;i<n;i++) { f[ct[i]][i] = a[i]; g[ct[i]][i
      ] = b[i]; }
  for(int i=0;i<=L;i++) { fwt(f[i], n, 1, 0, 1, 1); fwt
      (g[i], n, 1, 0, 1, 1); }
  for(int i=0;i<=L;i++)
    for(int j=0;j<=i;++j)
      for(int x=0;x<n;x++)
        add(h[i][x], mul(f[j][x], g[i-j][x]));
  for(int i=0;i<=L;i++) fwt(h[i], n, 1, 0, -1, 1);
  for(int i=0;i<n;i++) c[i] = h[ct[i]][i];
  return c;
}
```

7.4 Polynomial [5cfbc1]

```
// may need fac ivf
// == ab8066 ==
#define fi(s, n) for (int i = (int)(s); i < (int)(n); i
    ++)
#define neg(x) (x ? mod - x : 0)
#define V (*this)
template <int MAXN, int RT>  // MAXN = 2^k
struct Poly : vector<int> { // coefficients in [0, P)
  using vector<int>::vector;
  static inline NTT<MAXN, RT> ntt;
  int n() const { return (int)size(); }  // n() >= 1
  Poly(const Poly &p, int m) : vector<int>(m) { copy_n(
      p.data(), min(p.n(), m), data()); }
  Poly &irev() { return reverse(data(), data() + n()),
      V; }
  Poly &isz(int m) { return resize(m), V; }
// == e33faa ==
  Poly &iadd(const Poly &rhs) { // 1c6277
    fi(0, n()) add(V[i], rhs[i]);
    return V; // need n() == rhs.n()
  }
  Poly &imul(int k) { // 7e5b36
    fi(0, n()) V[i] = mul(V[i], k);
    return V;
  }
  Poly &mul_xk(int m){ // 19862a
    if(m<0) erase(begin(), begin()+(-m));
    else if(m>0) insert(begin(), m, 0);
    return V;
  }
  Poly Mul(const Poly &rhs) const { // ecd03e
    int m = 1;
    while (m < n() + rhs.n() - 1) m <<= 1;
    assert(m <= MAXN);
```

```
    Poly X(V, m), Y(rhs, m);
    ntt(X, m); ntt(Y, m);
    fi(0, m) X[i] = mul(X[i], Y[i]);
    ntt(X, m, true);
    return X.isz(n() + rhs.n() - 1);
  }
  Poly Inv() const { // 77f977
    if (n() == 1) return {inv(V[0])};
    int m = 1; // need V[0] != 0, 2*sz<=MAXN
    while (m < n() * 2) m <<= 1;
    assert(m <= MAXN);
    Poly Xi = Poly(V, (n() + 1) / 2).Inv().isz(m);
    Poly Y(V, m);
    ntt(Xi, m); ntt(Y, m);
    fi(0, m) {
      Xi[i] = mul(Xi[i], (2 - mul(Xi[i], Y[i])));
      add(Xi[i], mod);
    }
    ntt(Xi, m, true);
    return Xi.isz(n());
  }
// == 095701 ==
  Poly &shift_inplace(const int &c) { // need fac[],
      ivf[]
    int n = V.n(); // 2 * sz <= MAXN
    for (int i = 0; i < n; i++) V[i] = mul(V[i], fac[i
        ]);
    Poly g(n);
    int cp = 1;
    for (int i = 0; i < n; i++){
      g[i] = mul(cp, ivf[i]);
      cp = mul(cp, c);
    }
    V = V.irev().Mul(g).isz(n).irev();
    for (int i = 0; i < n; i++) V[i] = mul(V[i], ivf[i
        ]);
    return V;
  }
  Poly Shift(const int &c) const { return Poly(V).
      shift_inplace(c); }
// == 0bd61d ==
  Poly Dx() const {
    Poly ret(n() - 1);
    fi(0, ret.n()) ret[i] = mul(i+1, V[i+1]);
    return ret.isz(max(1, ret.n()));
  }
  Poly Sx() const {
    Poly ret(n() + 1);
    fi(0, n()) ret[i + 1] = mul(inv(i+1), V[i]);
    return ret;
  }
// == 10e23d ==
  Poly Ln() const {  // V[0] == 1, 2*sz<=MAXN
    return Dx().Mul(Inv()).Sx().isz(n());
  }
  Poly Exp() const {  // V[0] == 0,2*sz<=MAXN
    if (n() == 1) return {1};
    Poly X = Poly(V, (n() + 1) / 2).Exp().isz(n());
    Poly Y = X.Ln();
    Y[0] = mod - 1;
    fi(0, n()){
      Y[i] = V[i] - Y[i];
      add(Y[i], mod);
    }
    return X.Mul(Y).isz(n());
  }
// == eaf14c ==
  // M := P(P - 1). If k >= M, k := k % M + M.
  Poly Pow(ll k) const {  // 2*sz<=MAXN
    int nz = 0;
    while (nz < n() && !V[nz]) nz++;
    if (nz * min(k, (ll)n()) >= n()) return Poly(n());
    if (!k) return Poly(Poly{1}, n());
    Poly X(data() + nz, data() + nz + n() - nz * k);
    const int c = po(X[0], k % (mod - 1));
    return X.Ln().imul(k % mod).Exp().imul(c).irev().
        isz(n()).irev();
  }
// == cdf741 ==
  Poly _tmul(int nn, const Poly &rhs) const {
    Poly Y = Mul(rhs).isz(n() + nn - 1);
    return Poly(Y.data() + n() - 1, Y.data() + Y.n());
  }
  vector<int> _eval(const vector<int> &x, const vector<
      Poly> &up) const { // 82d6be
    const int m = (int)x.size();
    if (!m) return {};
    vector<Poly> down(m * 2);
    down[1] = Poly(up[1]).irev().isz(n()).Inv().irev().
        _tmul(m, V);
    fi(2, m * 2) down[i] = up[i ^ 1]._tmul(up[i].n() -
        1, down[i / 2]);
    vector<int> y(m);
    fi(0, m) y[i] = down[m + i][0];
    return y;
  }
  static vector<Poly> _tree1(const vector<int> &x) { //
      2a0b6b
    const int m = (int)x.size();
    vector<Poly> up(m * 2);
    fi(0, m) up[m + i] = {neg(x[i]), 1};
    for (int i = m - 1; i > 0; --i) up[i] = up[i * 2].
        Mul(up[i * 2 + 1]);
    return up;
  }
  vector<int> Eval(const vector<int> &x) const {
    auto up = _tree1(x); // 2^17, 1.8s
    return _eval(x, up);
  }
// == f9ecdc ==
  static Poly Interpolate(const vector<int> &x, const
      vector<int> &y) { // 8f2a08
    const int m = (int)x.size(); // 2^17, 2.3s
    vector<Poly> up = _tree1(x), down(m * 2);
    vector<int> z = up[1].Dx()._eval(x, up);
    fi(0, m) z[i] = mul(y[i], inv(z[i]));
    fi(0, m) down[m + i] = {z[i]};
    for (int i = m - 1; i > 0; --i)
      down[i] = down[i * 2].Mul(up[i * 2 + 1]).iadd(
          down[i * 2 + 1].Mul(up[i * 2]));
    return down[1];
  }
  pair<Poly, Poly> DivMod(const Poly &rhs) const { //
      a75170
    if (n() < rhs.n()) return {{0}, V}; // 5e5, 0.9s
    const int m = n() - rhs.n() + 1;
    Poly X(rhs); // (rhs.)back() != 0
    X.irev().isz(m);
    Poly Y(V); Y.irev().isz(m);
    Poly Q = Y.Mul(X.Inv()).isz(m).irev();
    X = rhs.Mul(Q), Y = V;
    fi(0, n()) add(Y[i], -X[i]);
    return {Q, Y.isz(max(1, rhs.n() - 1))};
  }
// == 7cd4c4 ==
  Poly _Sqrt() const {  // Jacobi(V[0], P) = 1
    if (n() == 1) return {QuadraticResidue(V[0], mod)};
    Poly X = Poly(V, (n() + 1) / 2)._Sqrt().isz(n());
    return X.iadd(Mul(X.Inv()).isz(n())).imul(mod / 2 +
        1);
  }
  Poly Sqrt() const { // 288427
    Poly a; // 2 * sz <= MAXN
    bool has = 0;
    for (int i = 0; i < n(); i++) {
      if (V[i]) has = 1;
      if (has) a.push_back(V[i]);
    }
    if (!has) return V;
    if ((n() + a.n()) % 2 || Jacobi(a[0], mod) != 1) {
      return Poly();
    }
    a = a.isz((n() + a.n()) / 2)._Sqrt();
    int sz = a.n();
    a.isz(n());
    rotate(a.begin(), a.begin() + sz, a.end());
    return a;
  }
// == 0a54cf ==
  Poly Shift_samples(int c, int m){
    // V = \sum f(n) x^n, return f(c), ..., f(c+m-1);
        2^19, 2s
    Poly A=V;
    Poly Q(n()+1), S(n());
```

```
    int nw=1;
    fi(0, n()+1) { Q[i] = mul(1-2*(i&1), nw); nw = mul(
        nw, n()-i, inv(i+1));}
    nw=1;
    fi(0, n()) { S[i] = mul(1-2*(i&1), nw); nw = mul(nw
        , c-i, inv(i+1));}
    S=S.Shift(1);
    fi(0,n()) if(i&1) {S[i] = mul(S[i],-1); add(S[i],
        mod);};
    Poly C=Mul(Q).mul_xk(-n());
    auto tmp=Q.isz(m).Inv();
    C=C.imul(-1).Mul(tmp).isz(m).mul_xk(n());
    A=A.isz(n()+m).iadd(C).irev().isz(n()+m);
    return A.Mul(S).mul_xk(-n()+1).isz(m+1).irev();
  }
// == 224b20 ==
  Poly power_projection(Poly wt, int m) { // 857237
    assert(n() == wt.n()); // 4*sz <= MAXN
    if (!n()) return Poly(m + 1, 0);
    if (V[0] != 0) {
      int c = V[0];
      V[0] = 0;
      Poly A = V.power_projection(wt, m);
      fi(0, m + 1) A[i] = mul(A[i], fac[i]);
      Poly B(m + 1);
      int pow = 1;
      fi(0, m + 1) {B[i] = mul(pow, ivf[i]); pow = mul(
          pow, c);}  // inv. of fac
      A = A.Mul(B).isz(m + 1);
      fi(0, m + 1) A[i] = mul(A[i], fac[i]);
      return A;
    }
    int n = 1;
    while (n < V.n()) n *= 2;
    isz(n), wt.isz(n).irev();
    int k = 1;
    Poly p(wt, 2 * n), q(V, 2 * n);
    q.imul(mod - 1);
    while (n > 1) {
      Poly r(2 * n * k);
      fi(0, 2 * n * k) r[i] = (i % 2 == 0 ? q[i] : neg(
          q[i]));
      Poly pq = p.Mul(r).isz(4 * n * k);
      Poly qq = q.Mul(r).isz(4 * n * k);
      fi(0, 2 * n * k) {
        add(pq[2 * n * k + i], p[i]);
        add(qq[2 * n * k + i], (q[i] + r[i]) % mod);
      }
      fill(p.begin(), p.end(), 0);
      fill(q.begin(), q.end(), 0);
      for(int j = 0; j < 2 * k; j++) fi(0, n / 2) {
        p[n * j + i] = pq[(2 * n) * j + (2 * i + 1)];
        q[n * j + i] = qq[(2 * n) * j + (2 * i + 0)];
      }
      n /= 2; k *= 2;
    }
    Poly ans(k);
    fi(0, k) ans[i] = p[2 * i];
    return ans.irev().isz(m + 1);
  }
  Poly FPSinv() { // 49e5ad
    const int n = V.n() - 1; // 2^17, 4s
    if (n == -1) return {};
    assert(V[0] == 0);
    if (n == 0) return V;
    assert(V[1] != 0);
    int c = V[1], ic = inv(c);
    imul(ic);
    Poly wt(n + 1);
    wt[n] = 1;
    Poly A = V.power_projection(wt, n);
    Poly g(n);
    fi(1, n + 1) g[n - i] = mul(n, A[i], inv(i));
    g = g.Pow(neg(inv(n)));
    g.insert(g.begin(), 0);
    int pow = 1;
    fi(0, g.n()) {g[i] = mul(g[i], pow); pow = mul(pow,
         ic);}
    return g;
  }
// == 71ee33 ==
  Poly TMul(const Poly &rhs) const { // this[i] - rhs[j
        ] = k;
    return Poly(V).irev().Mul(rhs).isz(n()).irev();
  }
  Poly comp_rec(int n, int k, Poly Q){ // 06ca07
    if (n == 1) {
      Poly p(2 * k);
      irev();
      fi(0, k) p[2 * i] = V[i];
      return p;
    }
    Poly R(2 * n * k);
    fi(0, 2 * n * k) R[i] = (i % 2 == 0 ? Q[i] : neg(Q[
        i]));
    Poly QQ = Q.Mul(R).isz(4 * n * k);
    fi(0, 2 * n * k)
      add(QQ[2 * n * k + i], (Q[i] + R[i]) % mod);
    Poly nxt_Q(2 * n * k);
    for(int j = 0; j < 2 * k; j++) fi(0, n / 2)
      nxt_Q[n * j + i] = QQ[(2 * n) * j + (2 * i + 0)];
    Poly nxt_p = comp_rec(n / 2, k * 2, nxt_Q);
    Poly pq(4 * n * k);
    for(int j = 0; j < 2 * k; j++) fi(0, n / 2)
      add(pq[(2 * n) * j + (2 * i + 1)], nxt_p[n * j +
          i]);
    Poly p(2 * n * k);
    fi(0, 2 * n * k) add(p[i], pq[2 * n * k + i]);
    pq.pop_back();
    Poly x = pq.TMul(R);
    fi(0, 2 * n * k) add(p[i], x[i]);
    return p;
  }
  Poly FPScomp(Poly g) { // solves V(g(x))
    int sz = 1; // 2^17, 5s
    while(sz < n() || sz < g.n()) sz <<= 1;
    return isz(sz), comp_rec(sz, 1, g.imul(mod-1).isz(2
         * sz)).isz(sz).irev();
  }
// == 1d60ad ==
};
#undef fi
#undef V
#undef neg
using Poly_t = Poly<1 << 21, 3>;
```

7.5 BM-BM [362501]

```
// out[i] = me[0]*out[i-1] + me[1]*out[i-2]+...
vector<int> BerlekampMassey(const vector<int> &output)
    { // 2e7709
  vector<int> d(output.size() + 1), me, he;
  for (int f = 0, i = 1; i <= sz(output); ++i) {
    for (int j = 0; j < sz(me); ++j)
      add(d[i], mul(output[i - j - 2], me[j]));
    add(d[i], -output[i-1]);
    if (d[i] == 0) continue;
    if (me.empty()) {
      me.resize(f = i);
      continue;
    }
    vector<int> o(i - f - 1);
    int k = mul(d[i], -inv(d[f]));
    o.push_back(-k);
    for (auto x : he) o.push_back(mul(x, k));
    if (o.size() < me.size()) o.resize(me.size());
    for (size_t j = 0; j < me.size(); ++j) add(o[j], me
        [j]);
    if (i-f+he.size() >= me.size()) he = me, f = i;
    me = o;
  }
  for(auto &h: me) add(h, mod);
  return me;
}

// Finds the k-th coefficient of P / Q in O(d log d log
     k)
// size of NTT has to > 2 * d
NTT<1<<19, 3> ntt;
int BostanMori(vector<int> P, vector<int> Q, long long
    k) { // 1bee33
  int d = max((int)P.size(), (int)Q.size() - 1);
  vector M = {P, Q};
  M[0].resize(d, 0);
```

```
  M[1].resize(d + 1, 0);
  int sz = (2 * d + 1 == 1 ? 2 : (1 << (__lg(2 * d) +
      1)));
  vector<int> Qn(sz);
  vector N(2, vector<int>(sz));
  while(k) {
    fill(iter(Qn), 0);
    for(int i = 0; i < d + 1; i++){
      Qn[i] = M[1][i] * ((i & 1) ? -1 : 1);
      add(Qn[i], mod);
    }
    ntt(Qn, sz, false);
    int t[2] = {(int)k & 1, 0};
    for(int i = 0; i < 2; i++){
      fill(iter(N[i]), 0);
      copy(iter(M[i]), N[i].begin());
      ntt(N[i], sz, false);
      for(int j = 0; j < sz; j++) N[i][j] = mul(N[i][j
          ], Qn[j]);
      ntt(N[i], sz, true);
      for(int j = t[i]; j < 2 * sz(M[i]); j += 2){
        M[i][j >> 1] = N[i][j];
      }
    }
    k >>= 1;
  }
  return mul(M[0][0], inv(M[1][0]));
}
// a_n = \sum_{j=1}^{d} c_j a_(n-j), c_0 = 0
int LinearRecursion(vector<int> a, vector<int> c, ll k)
    { // 3b1227
  int d = sz(a), sz = 1; // 1e5, 5s
  while(sz <= 2*d) sz <<= 1;
  c[0] = mod - 1;
  for(auto &i : c) i = i ? mod - i : 0;
  auto A = a; A.resize(sz);
  auto C = c; C.resize(sz);
  ntt(A, sz, false), ntt(C, sz, false);
  for(int i = 0; i < sz; i++) A[i] = mul(A[i], C[i]);
  ntt(A, sz, true);
  A.resize(d);
  return BostanMori(A, c, k);
}
```

7.6 Generating Functions

- Ordinary Generating Function $A(x) = \sum_{i\ge0} a_i x^i$
  - $A(rx) \Rightarrow r^n a_n$
  - $A(x) + B(x) \Rightarrow a_n + b_n$
  - $A(x)B(x) \Rightarrow \sum_{i=0}^{n} a_i b_{n-i}$
  - $A(x)^k \Rightarrow \sum_{i_1+i_2+\cdots+i_k=n} a_{i_1} a_{i_2} \dots a_{i_k}$
  - $xA(x)' \Rightarrow n a_n$
  - $\frac{A(x)}{1-x} \Rightarrow \sum_{i=0}^{n} a_i$
- Exponential Generating Function $A(x) = \sum_{i\ge0} \frac{a_i}{i!} x_i$
  - $A(x) + B(x) \Rightarrow a_n + b_n$
  - $A^{(k)}(x) \Rightarrow a_{n+k}$
  - $A(x)B(x) \Rightarrow \sum_{i=0}^{n} \binom{n}{i} a_i b_{n-i}$
  - $A(x)^k \Rightarrow \sum_{i_1+i_2+\cdots+i_k=n} \binom{n}{i_1,i_2,\dots,i_k} a_{i_1} a_{i_2} \dots a_{i_k}$
  - $xA(x) \Rightarrow n a_n$
- Special Generating Function
  - $(1+x)^n = \sum_{i\ge0} \binom{n}{i} x^i$
  - $\frac{1}{(1-x)^n} = \sum_{i\ge0} \binom{n+i-1}{n-1} x^i$
- Catalan number: $f(x) = \frac{1-\sqrt{1-4x}}{2x}$
- Suppose $A(x)$ is the EGF of a labelled component. Then $B(x) = e^{A(x)}$ is the EGF of the same thing but with any number of components.
- Lagrange's Inversion Formula

  $F(G(x)) = G(F(x)) = x$, then $F(0) = G(0) = 0$ and $F'(0), G'(0) \neq 0$. Let H be any FPS, then

  $$n[x^n]F(x) = [x^{n-1}]H'(x)\left(\frac{x}{G(x)}\right)^n.$$
- Newton's Method

  Suppose FPS P satisfies $F(x, P(x)) = 0$ where $F(x, y)$ is some polynomial, then

  $$P_{2n}(x) = P_n(x) - F(x, P_n(x)) / \frac{\partial F}{\partial y}(x, P_n(x)).$$

8 Geometry

8.1 Basic [d4b33e]

```
// == d2c7dd ==
struct PT {
  ll x, y;
  auto operator<=>(const PT&) const = default;
};
PT operator+(PT p1, PT p2)
{ return PT{p1.x + p2.x, p1.y + p2.y}; }
PT operator-(PT p1, PT p2)
{ return PT{p1.x - p2.x, p1.y - p2.y}; }
ll operator*(PT p1, PT p2)
{ return p1.x * p2.x + p1.y * p2.y; }
ll operator^(PT p1, PT p2)
{ return p1.x * p2.y - p1.y * p2.x; }
int sign(ll a) { return a == 0 ? 0 : a > 0 ? 1 : -1; }
int ori(PT a, PT b, PT c) { // is C to the left of A->B
  return sign((b-a)^(c-a)); }
bool btw(PT a, PT b, PT c) { // is C between AB
  return !ori(a, b, c) && sign((a-c)*(b-c)) <= 0; }
PT ccw90(PT p) { return PT(-p.y, p.x); }
// == f61f45 ==
// change PT::x, y to FP type, replace sign()
// needs PT+-PT, PT*/scalar
const double EPS = 1e-9;
int sign(double a) { return abs(a)<EPS?0:a>0?1:-1; }
double abs2(PT p){ return p * p; }
double abs(PT p){ return sqrt(p * p); }
PT proj(PT a, PT b, PT c){ // C projected to AB
  return a+(b-a)*((c-a)*(b-a)/abs2(b-a));}
double dist(PT a,PT b,PT c){ // distance from C to AB
  return abs((c-a)^(b-a))/abs(b-a);}
// struct Line{ PT a, b; };
// struct Cir{ PT o; double r; };
```

8.2 Convex Hull [903417]

```
// (int)
// requires sz(pt) >= 2 & distinct points
// points on edges are excluded
vector<PT> ConvexHull(vector<PT> pt) {
  int n = sz(pt);
  sort(iter(pt));
  vector<PT> ans{pt[0]};
  For(t, 0, 1) {
    int m = sz(ans);
    For(i, 1, n - 1) {
      while (sz(ans) > m && ori(ans[sz(ans) - 2],
        ans.back(), pt[i]) <= 0) ans.pop_back();
      ans.push_back(pt[i]);
    }
    reverse(iter(pt));
  }
  if (sz(ans) > 1) ans.pop_back();
  return ans;
}
```

8.3 Dynamic Convex Hull [95890b]

```
// (int)
struct DynamicConvexHull {
  struct UpCmp {
    bool operator()(PT a, PT b) const {
      return a < b;
    }
  };
  struct DownCmp {
    bool operator()(PT a, PT b) const {
      return a > b;
    }
  };
  template <typename T>
  struct Hull {
    set<PT,T> hull;
    bool chk(PT i, PT j, PT k) {
      return ((k-i)^(j-i)) > 0;
    }
    void insert(PT x) {
      if(inside(x)) return;
```

```
      hull.insert(x);
      auto it=hull.lower_bound(x);
      if(next(it)!=hull.end()) {
        for (auto it2 = next(it);
            next(it2) != hull.end(); ++it2) {
          if(chk(x,*it2,*next(it2))) break;
          hull.erase(it2);
          it2=hull.lower_bound(x);
        }
      }
      it=hull.lower_bound(x);
      if(it!=hull.begin()) {
        for (auto it2 = prev(it);
            it2 != hull.begin(); --it2) {
          if(chk(*prev(it2),*it2,x)) break;
          hull.erase(it2);
          it2=hull.lower_bound(x);
          if(it2==hull.begin()) break;
        }
      }
    }
    bool inside(PT x) {
      if (hull.lower_bound(x) != hull.end() &&
          *hull.lower_bound(x) == x)
        return true;
      auto it=hull.lower_bound(x);
      bool ans=false;
      if(it!=hull.begin()&&it!=hull.end()) {
        ans=!chk(*prev(it),x,*it);
      }
      return ans;
    }
  };
  Hull<UpCmp> up;
  Hull<DownCmp> down;
  void insert(PT x){up.insert(x),down.insert(x);}
  bool inside(PT x) {
    return up.inside(x) && down.inside(x);
  }
};
```

8.4 Point In Convex Hull [9a1f2c]

```
// (int)
// C must be a convex polygon in cyclic order
bool PointInConvex(const vector<PT> &C, PT p,
                   bool strict = true) {
  int a = 1, b = int(C.size()) - 1, r = !strict;
  if (C.size() == 0) return false;
  if (C.size() < 3) return r && btw(C[0], C.back(), p);
  if (ori(C[0], C[a], C[b]) > 0) swap(a, b);
  if (ori(C[0], C[a], p) >= r ||
      ori(C[0], C[b], p) <= -r) return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (ori(C[0], C[c], p) > 0 ? b : a) = c;
  }
  return ori(C[a], C[b], p) < r;
}
```

8.5 Point In Circle [9180b6]

```
// (int)
// p1, p2, p3 must be in CCW order
// is p4 strictly in circumcircle of p1p2p3
ll sq(ll x) { return x * x; }
bool InCir(PT p1, PT p2, PT p3, PT p4) {
  ll u11 = p1.x - p4.x; ll u12 = p1.y - p4.y;
  ll u21 = p2.x - p4.x; ll u22 = p2.y - p4.y;
  ll u31 = p3.x - p4.x; ll u32 = p3.y - p4.y;
  ll u13 = sq(p1.x) - sq(p4.x) + sq(p1.y) - sq(p4.y);
  ll u23 = sq(p2.x) - sq(p4.x) + sq(p2.y) - sq(p4.y);
  ll u33 = sq(p3.x) - sq(p4.x) + sq(p3.y) - sq(p4.y);
  i128 det =
    - (i128)u13 * u22 * u31 + (i128)u12 * u23 * u31
    + (i128)u13 * u21 * u32 - (i128)u11 * u23 * u32
    - (i128)u12 * u21 * u33 + (i128)u11 * u22 * u33;
  return det > 0;
}
```

8.6 Half Plane Intersection [c77d62]

```
// (int) needs polar angle
// arr must be nonempty; keeps the left of each line
auto area_pair(Line a, Line b) {
  return make_pair((a.b - a.a) ^ (b.a - a.a),
    (a.b - a.a) ^ (b.b - a.a));
}
bool isin(Line l0, Line l1, Line l2) {
  // Check inter(l1, l2) strictly in l0
  auto [a02X, a02Y] = area_pair(l0, l2);
  auto [a12X, a12Y] = area_pair(l1, l2);
  if (a12X < a12Y) a12X *= -1, a12Y *= -1;
  return (i128)a02Y * a12X > (i128)a02X * a12Y;
}
/* [solution exists] <=> [result.size() > 2] */
/* --^-- Line.a --^-- Line.b --^-- */
vector<Line> HalfPlaneInter(vector<Line> arr) {
  sort(arr.begin(), arr.end(), [&](Line a, Line b) {
    PT p1 = a.b - a.a, p2 = b.b - b.a;
    if (polar(p1, p2) != 0) return polar(p1, p2) < 0;
    return ori(a.a, a.b, b.b) < 0;
  });
  deque<Line> dq(1, arr[0]);
  auto pop_back = [&](int t, Line p) {
    while (sz(dq) >= t &&
        !isin(p, dq[sz(dq) - 2], dq.back()))
      dq.pop_back();
  };
  auto pop_front = [&](int t, Line p) {
    while (sz(dq) >= t && !isin(p, dq[0], dq[1]))
      dq.pop_front();
  };
  for (auto p : arr)
    if (polar(dq.back().b - dq.back().a,
              p.b - p.a) != 0)
      pop_back(2, p), pop_front(2, p),
      dq.push_back(p);
  pop_back(3, dq[0]), pop_front(3, dq.back());
  return vector<Line>(dq.begin(), dq.end());
}
```

8.7 Minkowski Sum [f559ee]

```
// (int) needs convex hull
// each input needs at least two distinct points
vector<PT> Minkowski(vector<PT> a, vector<PT> b) {
  a = ConvexHull(a), b = ConvexHull(b);
  int n = sz(a), m = sz(b);
  vector<PT> c{a[0] + b[0]}, s1, s2;
  For(i, 0, n - 1)
    s1.push_back(a[(i + 1) % n] - a[i]);
  For(i, 0, m - 1)
    s2.push_back(b[(i + 1) % m] - b[i]);
  for (int p1 = 0, p2 = 0; p1 < n || p2 < m;)
    if (p2 == m || (p1 < n &&
        sign(s1[p1] ^ s2[p2]) >= 0))
      c.push_back(c.back() + s1[p1++]);
    else c.push_back(c.back() + s2[p2++]);
  return ConvexHull(c);
}
```

8.8 Polar Angle [423cf8]

```
// (int)
// CCW from (1, 0), without length tie-breaking
int halfplane(PT p) {
  if (sign(p * p) == 0) return 0;
  bool upper = sign(p.y) > 0 ||
    (sign(p.y) == 0 && sign(p.x) > 0);
  return 1 - 2 * upper;
}  // upper(-1) -> origin(0) -> lower(1)
strong_ordering polar(PT a, PT b) {
  int ha = halfplane(a), hb = halfplane(b);
  if (ha != hb) return ha <=> hb;
  return 0 <=> sign(a ^ b);
}
```

8.9 Rotating Sweep Line [f1c4b0]

```
// (int) needs polar angle
// pts: 0-indexed Pt array
void RotSwpLine(int n, PT* pts) {
  using E = pair<PT, pii>;
```

```
  vector<E> ev;  // dir, i, j: (i, j)=>(j, i)
  For(i, 0, n - 1) For(j, 0, i - 1) {
    PT dir = pts[j] - pts[i];
    halfplane(dir) < 0 ? ev.push_back({dir, {i, j}})
      : ev.push_back({PT(0, 0) - dir, {j, i}});
  }
  sort(ev.begin(), ev.end(), [&](E e1, E e2) {
    auto pol = polar(e1.F, e2.F);
    return pol < 0 || (pol == 0 &&
        pll(e1.F * pts[e1.S.F], e1.F * pts[e1.S.S])
      < pll(e1.F * pts[e2.S.F], e1.F * pts[e2.S.S]));
  });
  vector<int> ord(n), rk(n);
  iota(ord.begin(), ord.end(), 0);
  sort(ord.begin(), ord.end(), [&](int i, int j) {
    return make_pair(pts[i].y, pts[i].x)
      < make_pair(pts[j].y, pts[j].x);
  });
  For(i, 0, n - 1) rk[ord[i]] = i;
  // init order with point indices ord[*]
  int ne = (int)ev.size();
  For(ie, 0, ne - 1) {
    int i, j; tie(i, j) = ev[ie].S;
    // update swap i, j
    rk[i]++; rk[j]--;   // rk[point idx]: rank in ord
    tie(ord[rk[i]], ord[rk[i] - 1]) = tie(i, j);
    if (ie == ne - 1 ||
        polar(ev[ie + 1].F, ev[ie].F) != 0) {
      // update answer
    }
  }
}
```

8.10 Segment Intersect [1a6a88]

```
bool HasInter(PT p1, PT p2, PT p3, PT p4) { // int //
    e7c1d1
  int a123 = ori(p1, p2, p3);
  int a124 = ori(p1, p2, p4);
  int a341 = ori(p3, p4, p1);
  int a342 = ori(p3, p4, p2);
  if (!a123 && !a124)
    return btw(p1, p2, p3) || btw(p1, p2, p4) ||
      btw(p3, p4, p1) || btw(p3, p4, p2);
  return a123 * a124 <= 0 && a341 * a342 <= 0;
}  // does p1p2 intersect p3p4
PT GetInter(PT p1, PT p2, PT p3, PT p4) { // FP // 2629
    b1
  double a123 = (p2 - p1) ^ (p3 - p1);
  double a124 = (p2 - p1) ^ (p4 - p1);
  return (p4 * a123 - p3 * a124) / (a123 - a124);
}  // C^3 / C^2; requires nonparallel inputs
```

8.11 Circle Intersect With Any [c31e48]

```
// (FP) nondegenerate lines; polygon edges != 0
vector<PT> CircleLineInter(Cir c, Line l) { // ab01ed
  PT p = l.a + (l.b - l.a) *
    ((c.o - l.a) * (l.b - l.a)) /
    abs2(l.b - l.a);
  double s = (l.b - l.a) ^ (c.o - l.a);
  double h2 = c.r * c.r - s * s /
    abs2(l.b - l.a);
  if (sign(h2) == -1) return {};
  if (sign(h2) == 0) return {p};
  PT h = (l.b - l.a) / abs(l.b - l.a) * sqrt(h2);
  return {p - h, p + h};
}
vector<PT> CirclesInter(Cir c1, Cir c2) { // e9fe3a
  double d2 = abs2(c1.o - c2.o), d = sqrt(d2);
  if (sign(d2) == 0) return {}; // concentric circles
  if (d < max(c1.r, c2.r) - min(c1.r, c2.r) ||
      d > c1.r + c2.r) return {};
  PT u = (c1.o + c2.o) / 2 + (c1.o - c2.o) *
    ((c2.r * c2.r - c1.r * c1.r) / (2 * d2));
  double A = sqrt((c1.r + c2.r + d) *
    (c1.r - c2.r + d) * (c1.r + c2.r - d) *
    (-c1.r + c2.r + d));
  PT v = PT{c1.o.y - c2.o.y, -c1.o.x + c2.o.x}
    * A / (2 * d2);
  if (sign(v.x) == 0 && sign(v.y) == 0) return {u};
  return {u + v, u - v};
}
double _area(PT pa, PT pb, double r) { // 1238aa
  if (abs(pa) < abs(pb)) swap(pa, pb);
  if (abs(pb) < EPS) return 0;
  double S, h, theta;
  double a = abs(pb), b = abs(pa), c = abs(pb - pa);
  double cosB = pb * (pb - pa) / a / c, B = acos(cosB);
  double cosC = (pa * pb) / a / b, C = acos(cosC);
  if (a > r) {
    S = (C / 2) * r * r;
    h = a * b * sin(C) / c;
    if (h < r && B < numbers::pi / 2)
      S -= acos(h / r) * r * r -
        h * sqrt(r * r - h * h);
  } else if (b > r) {
    theta = numbers::pi - B - asin(sin(B) / r * a);
    S = .5 * a * r * sin(theta) +
      (C - theta) / 2 * r * r;
  } else S = .5 * sin(C) * a * b;
  return S;
}
double AreaPolyCircle(vector<PT> poly, PT O,
    double r) { // 934b82
  double S = 0; int n = sz(poly);
  for (int i = 0; i < n; ++i)
    S += _area(poly[i] - O, poly[(i + 1) % n] - O,
      r) * ori(O, poly[i], poly[(i + 1) % n]);
  return fabs(S);
}
```

8.12 Tangents [b087af]

```
// (FP) circles have positive radii
PT unit(PT p) { return p / abs(p); }
PT ccw(PT p, double t) {
  return PT(
    p.x * cos(t) - p.y * sin(t),
    p.x * sin(t) + p.y * cos(t)
  );
}
vector<Line> Tangent(Cir c, PT p) { // bb4a23
  vector<Line> z;
  double d = abs(p - c.o);
  if (sign(d - c.r) == 0) {
    z.push_back({p, p + ccw90(p - c.o)});
  } else if (d > c.r) {
    double o = acos(c.r / d);
    PT i = unit(p - c.o);
    PT j = ccw(i, o) * c.r;
    PT k = ccw(i, -o) * c.r;
    z.push_back({c.o + j, p});
    z.push_back({c.o + k, p});
  }
  return z;
}
vector <Line> Tangent(Cir c1, Cir c2, int sign1) { //
    d97c9a
  // sign1 = 1 for outer, -1 for inter
  vector <Line> ret;
  double d_sq = abs2(c1.o - c2.o);
  if (sign(d_sq) == 0) return ret;
  double d = sqrt(d_sq);
  PT v = (c2.o - c1.o) / d;
  double c = (c1.r - sign1 * c2.r) / d;
  if (c * c > 1) return ret;
  double h = sqrt(max(0.0, 1.0 - c * c));
  for (int sign2 = 1; sign2 >= -1; sign2 -= 2) {
    PT n = PT{v.x * c - sign2 * h * v.y,
      v.y * c + sign2 * h * v.x};
    PT p1 = c1.o + n * c1.r;
    PT p2 = c2.o + n * (c2.r * sign1);
    if (sign(p1.x - p2.x) == 0 &&
        sign(p1.y - p2.y) == 0)
      p2 = p1 + ccw90(c2.o - c1.o);
    ret.push_back({p1, p2});
  }
  return ret;
}
```

8.13 Tangent to Convex Hull [f1fdc2]

```
// (int) needs cyclic ternary search
```

```
// C is nonempty; p is strictly outside C
pii HullTangent(vector<PT> &C, PT p) {
  auto gao = [&](int s) {
    return cyc_tsearch(sz(C), [&](int x, int y)
    { return ori(p, C[x], C[y]) == s; });
  };
  return pii(gao(1), gao(-1));
} // return (a, b), ori(p, C[a], C[b]) >= 0
```

8.14 Minimum Enclosing Circle [5d2e56]

```
// (FP) needs segment intersect
// p must be nonempty
PT circenter(PT a, PT b, PT c) {
  PT ab = (a + b) / 2, ac = (a + c) / 2;
  return GetInter(
    ab, ab + ccw90(b - a),
    ac, ac + ccw90(c - a)
  );
}
Cir MinEnclosing(vector<PT> &p) {
  shuffle(p.begin(), p.end(), mt19937(clock()));
  double r = 0;
  PT cent = p[0];
  For(i, 0, sz(p) - 1)
  if (abs2(cent - p[i]) > r) {
    cent = p[i]; r = 0;
    For(j, 0, i - 1)
    if (abs2(cent - p[j]) > r) {
      cent = (p[i] + p[j]) / 2;
      r = abs2(p[j] - cent);
      For(k, 0, j - 1)
      if (abs2(cent - p[k]) > r) {
        cent = circenter(p[i], p[j], p[k]);
        r = abs2(p[k] - cent);
      }
    }
  }
  return {cent, sqrt(r)};
}
```

8.15 Union of Stuff [ae9b12]

```
// (FP) needs segment intersect
// circles are pairwise distinct; polygons are CCW
vector<pair<double, double>> CoverSegment(Cir a, Cir b)
     { // 7ae1be
  double d = abs(a.o - b.o);
  vector<pair<double, double>> res;
  if (sign(a.r + b.r - d) == 0);
  else if (d <= abs(a.r - b.r) + EPS) {
    if (a.r < b.r)
      res.emplace_back(0, 2 * numbers::pi);
  } else if (d < abs(a.r + b.r) - EPS) {
    double o = acos((a.r * a.r + d * d - b.r * b.r)
      / (2 * a.r * d));
    double z = atan2((b.o - a.o).y, (b.o - a.o).x);
    if (z < 0) z += 2 * numbers::pi;
    double l = z - o, r = z + o;
    if (l < 0) l += 2 * numbers::pi;
    if (r > 2 * numbers::pi) r -= 2 * numbers::pi;
    if (l > r)
      res.emplace_back(l, 2 * numbers::pi),
      res.emplace_back(0, r);
    else res.emplace_back(l, r);
  }
  return res;
}
double CircleUnionArea(vector<Cir> c) { // e653e9
  int n = sz(c);
  double a = 0, w;
  for (int i = 0; w = 0, i < n; ++i) {
    vector<pair<double, double>> s = {
      {2 * numbers::pi, 9}
    }, z;
    for (int j = 0; j < n; ++j) if (i != j) {
      z = CoverSegment(c[i], c[j]);
      for (auto &e : z) s.push_back(e);
    }
    sort(s.begin(), s.end());
    auto F = [&] (double t) {
      return c[i].r * (c[i].r * t +
        c[i].o.x * sin(t) - c[i].o.y * cos(t));
    };
    for (auto &e : s) {
      if (e.first > w) a += F(e.first) - F(w);
      w = max(w, e.second);
    }
  }
  return a * 0.5;
}
// Union of Polygons
double PolyUnion(vector<vector<PT>> poly) { // 827711
  int n = sz(poly);
  double ans = 0;
  auto solve = [&](PT a, PT b, int cid) {
    vector<pair<PT, int>> event;
    for (int i = 0; i < n; ++i) {
      int st = 0, m = sz(poly[i]);
      while (st < m && ori(poly[i][st], a, b) != 1)
        st++;
      if (st == m) continue;
      For(j, 0, m - 1) {
        PT c = poly[i][(j + st) % m];
        PT d = poly[i][(j + st + 1) % m];
        if (sign((a - b) ^ (c - d)) != 0) {
          int ok1 = ori(c, a, b) == 1;
          int ok2 = ori(d, a, b) == 1;
          if (ok1 ^ ok2)
            event.emplace_back(GetInter(a, b, c, d),
              ok1 ? 1 : -1);
        } else if (ori(c, a, b) == 0 &&
            sign((a - b) * (c - d)) > 0 && i <= cid) {
          event.emplace_back(c, -1);
          event.emplace_back(d, 1);
        }
      }
    }
    sort(iter(event), [&](pair<PT, int> i,
                          pair<PT, int> j) {
      return (a - i.first) * (a - b) <
        (a - j.first) * (a - b);
    });
    int now = 0;
    PT lst = a;
    for (auto [x, y] : event) {
      if (btw(a, b, lst) && btw(a, b, x) && !now)
        ans += lst ^ x;
      now += y, lst = x;
    }
  };
  For(i, 0, n - 1) For(j, 0, sz(poly[i]) - 1)
    solve(poly[i][j], poly[i][(j + 1) % sz(poly[i])],
      i);
  return ans / 2;
}
```

8.16 Delaunay Triangulation [1c018f]

```
// (int) needs point in circle, segment intersect
const int N = 0; // set per problem constraints
// needs at least one distinct point
/* Delaunay Triangulation:
Given a sets of points on 2D plane, find a
triangulation such that no points will strictly
inside circumcircle of any triangle. */
struct Edge {
  int id; // oidx[id]
  list<Edge>::iterator twin;
  Edge(int _id = 0):id(_id) {}
};
struct Delaunay { // 0-base
  int n, oidx[N];
  list<Edge> head[N]; // result udir. graph
  PT p[N];
  void init(int _n, PT _p[]) {
    n = _n, iota(oidx, oidx + n, 0);
    for (int i = 0; i < n; ++i) head[i].clear();
    sort(oidx, oidx + n, [&](int a, int b) {
      return _p[a] < _p[b];
    });
    for (int i = 0; i < n; ++i) p[i] = _p[oidx[i]];
    divide(0, n - 1);
  }
```

```
  void addEdge(int u, int v) {
    head[u].push_front(Edge(v));
    head[v].push_front(Edge(u));
    head[u].begin()->twin = head[v].begin();
    head[v].begin()->twin = head[u].begin();
  }
  void divide(int l, int r) {
    if (l == r) return;
    if (l + 1 == r) return addEdge(l, l + 1);
    int mid = (l + r) >> 1, nw[2] = {l, r};
    divide(l, mid), divide(mid + 1, r);
    auto gao = [&](int t) {
      PT pt[2] = {p[nw[0]], p[nw[1]]};
      for (auto it : head[nw[t]]) {
        int v = ori(pt[1], pt[0], p[it.id]);
        if (v > 0 || (v == 0 &&
            (pt[t ^ 1] - p[it.id]) *
            (pt[t ^ 1] - p[it.id]) <
            (pt[1] - pt[0]) * (pt[1] - pt[0])))
          return nw[t] = it.id, true;
      }
      return false;
    };
    while (gao(0) || gao(1));
    addEdge(nw[0], nw[1]); // add tangent
    while (true) {
      PT pt[2] = {p[nw[0]], p[nw[1]]};
      int ch = -1, sd = 0;
      for (int t = 0; t < 2; ++t)
          for (auto it : head[nw[t]])
              if (ori(pt[0], pt[1], p[it.id]) > 0 &&
            (ch == -1 || InCir(pt[0], pt[1],
                    p[ch], p[it.id])))
                  ch = it.id, sd = t;
      if (ch == -1) break; // upper common tangent
      for (auto it = head[nw[sd]].begin();
           it != head[nw[sd]].end(); )
        if (HasStrictInter(pt[sd], p[it->id],
            pt[sd ^ 1], p[ch]))
          head[it->id].erase(it->twin),
          head[nw[sd]].erase(it++);
        else ++it;
      nw[sd] = ch, addEdge(nw[0], nw[1]);
    }
  }
} tool;
```

8.17 Voronoi Diagram [c6bcb2]

```
// (int) needs delaunay triangulation
// all coord is even, may call HalfPlaneInter after
vector<vector<Line>> vec;
void BuildVoronoiLine(int n, PT *arr) {
  tool.init(n, arr); // Delaunay
  vec.clear(), vec.resize(n);
  for (int i = 0; i < n; ++i)
    for (auto e : tool.head[i]) {
      int u = tool.oidx[i], v = tool.oidx[e.id];
      PT m{(arr[v].x + arr[u].x) / 2,
        (arr[v].y + arr[u].y) / 2};
      PT d = ccw90(arr[v] - arr[u]);
      vec[u].push_back({m, m + d});
    }
}
```

8.18 3D Basic [4a7a37]

```
// (FP) add Point*scalar for proj
struct Point { double x = 0, y = 0, z = 0; };
Point operator-(Point p1, Point p2)
{return Point{p1.x-p2.x, p1.y-p2.y, p1.z-p2.z};}
Point operator+(Point p1, Point p2)
{return Point{p1.x+p2.x, p1.y+p2.y, p1.z+p2.z};}
Point operator/(Point p1, double v)
{ return Point{p1.x / v, p1.y / v, p1.z / v}; }
Point operator^(Point p1, Point p2) {
  return Point{p1.y * p2.z - p1.z * p2.y,
    p1.z * p2.x - p1.x * p2.z,
    p1.x * p2.y - p1.y * p2.x};
}
double operator*(Point p1, Point p2)
{ return p1.x * p2.x + p1.y * p2.y + p1.z * p2.z; }
double abs(Point a) { return sqrt(a * a); }
Point cross3(Point a, Point b, Point c)
{ return (b - a) ^ (c - a); }
double area2(Point a, Point b, Point c)
{ return abs(cross3(a, b, c)); }
double volume(Point a, Point b, Point c, Point d)
{ return cross3(a, b, c) * (d - a); }
Point Masscenter(Point a, Point b, Point c, Point d)
{ return (a + b + c + d) / 4; }
PT proj(Point a, Point b, Point c, Point u) {
// proj. u to the plane of a, b, and c
  Point e1 = b - a;
  Point e2 = c - a;
  e1 = e1 / abs(e1);
  e2 = e2 - e1 * (e2 * e1);
  e2 = e2 / abs(e2);
  Point p = u - a;
  return PT{p * e1, p * e2};
}
```

8.19 3D Convex Hull [166108]

```
// (FP) needs 3D basic
// add Point==Point; returns {} unless hull is 3D
struct Face {
  int a, b, c;
  Face(int _a, int _b, int _c) : a(_a), b(_b), c(_c) {}
};
auto preprocess(auto pts) {
  auto G = pts.begin();
  vector<int> id;
  auto fail = tuple{-1, -1, -1, id};
  int a = int(find_if(iter(pts), [&](Point z) {
    return z != *G;
  }) - G);
  if (a == sz(pts)) return fail;
  int b = int(find_if(iter(pts), [&](Point z) {
    return cross3(*G, pts[a], z) != Point{0, 0, 0};
  }) - G);
  if (b == sz(pts)) return fail;
  int c = int(find_if(iter(pts), [&](Point z) {
    return sign(volume(*G, pts[a], pts[b], z)) != 0;
  }) - G);
  if (c == sz(pts)) return fail;
  for (int i = 0; i < sz(pts); i++)
    if (i != a && i != b && i != c)
      id.push_back(i);
  return tuple{a, b, c, id};
}
// return the faces with pts indexes
vector<Face> ConvexHull3D(vector<Point> pts) {
  int n = sz(pts);
  if (n <= 3) return {}; // be careful about edge case
  vector<Face> now;
  vector<vector<int>> z(n, vector<int>(n));
  auto [a, b, c, ord] = preprocess(pts);
  if (a == -1) return {};
  now.emplace_back(a, b, c); now.emplace_back(c, b, a);
  for (auto i : ord) {
    vector<Face> nxt;
    for (auto &f : now) {
      auto v = volume(pts[f.a], pts[f.b],
        pts[f.c], pts[i]);
      if (sign(v) <= 0) nxt.push_back(f);
      z[f.a][f.b] = z[f.b][f.c] =
        z[f.c][f.a] = sign(v);
    }
    auto F = [&](int x, int y) {
      if (z[x][y] > 0 && z[y][x] <= 0)
        nxt.emplace_back(x, y, i);
    };
    for (auto &f : now)
      F(f.a, f.b), F(f.b, f.c), F(f.c, f.a);
    now = nxt;
  }
  return now;
}
// n^2 delaunay: facets with negative z normal of
// convexhull of (x, y, x^2 + y^2), use a pseudo-point
// (0, 0, inf) to avoid degenerate case
// test @ SPOJ CH3D
// double area = 0, vol = 0; // surface area / volume
```

```
// for (auto [a, b, c]: faces)
//   area += abs(ver(p[a], p[b], p[c]))/2.0,
//   vol += volume(P3(0, 0, 0), p[a], p[b], p[c])/6.0;
```

9 Misc

9.1 All LCS [ba9cc9]

```
void all_lcs(string s, string t) { // 0-base
  vector<int> h(sz(t));
  iota(all(h), 0);
  for (int a = 0; a < sz(s); ++a) {
    int v = -1;
    for (int c = 0; c < sz(t); ++c)
      if (s[a] == t[c] || h[c] < v)
        swap(h[c], v);
    // LCS(s[0, a], t[b, c]) =
    // c - b + 1 - sum([h[i] >= b] | i <= c)
    // h[i] might become -1 !!
  }
}
```

9.2 Binary Search On Fraction [765c5a]

```
struct Q {
  ll p, q;
  Q go(Q b, ll d) { return {p + b.p*d, q + b.q*d}; }
};
bool pred(Q);
// returns smallest p/q in [lo, hi] such that
// pred(p/q) is true, and 0 <= p,q <= N
Q frac_bs(ll N) {
  Q lo{0, 1}, hi{1, 0};
  if (pred(lo)) return lo;
  assert(pred(hi));
  bool dir = 1, L = 1, H = 1;
  for (; L || H; dir = !dir) {
    ll len = 0, step = 1;
    for (int t = 0; t < 2 && (t ? step/=2 : step*=2);)
      if (Q mid = hi.go(lo, len + step);
          mid.p > N || mid.q > N || dir ^ pred(mid))
        t++;
      else len += step;
    swap(lo, hi = hi.go(lo, len));
    (dir ? L : H) = !!len;
  }
  return dir ? hi : lo;
}
```

9.3 Cyclic Ternary Search [9017cc]

```
/* bool pred(int a, int b);
f(0) ~ f(n - 1) is a cyclic-shift U-function
return idx s.t. pred(x, idx) is false forall x*/
int cyc_tsearch(int n, auto pred) {
  if (n == 1) return 0;
  int l = 0, r = n; bool rv = pred(1, 0);
  while (r - l > 1) {
    int m = (l + r) / 2;
    if (pred(0, m) ? rv: pred(m, (m + 1) % n)) r = m;
    else l = m;
  }
  return pred(l, r % n) ? l : r % n;
}
```

9.4 Matroid Intersection

$M = (E, \mathcal{I})$, where $\mathcal{I} \subseteq 2^E$ is nonempty, is a matroid if:

- If $S \in \mathcal{I}$ and $S' \subsetneq S$, then $S' \in \mathcal{I}$.
- For $S_1, S_2 \in \mathcal{I}$ s.t. $|S_1| < |S_2|$, there exists $e \in S_2 \setminus S_1$ s.t. $S_1 \cup \{e\} \in \mathcal{I}$.

Matroid intersection:
Start from $S = \emptyset$. In each iteration, let

- $Y_1 = \{x \notin S \mid S \cup \{x\} \in \mathcal{I}_1\}$
- $Y_2 = \{x \notin S \mid S \cup \{x\} \in \mathcal{I}_2\}$

If there exists $x \in Y_1 \cap Y_2$, insert x into S. Otherwise for each $x \in S, y \notin S$, create edges

- $x \to y$ if $S - \{x\} \cup \{y\} \in \mathcal{I}_1$.
- $y \to x$ if $S - \{x\} \cup \{y\} \in \mathcal{I}_2$.

Find a *shortest* path (with BFS) starting from a vertex in Y_1 and ending at a vertex in Y_2 which doesn't pass through any other vertices in Y_2, and alternate the path. The size of S will be incremented by 1 in each iteration. For the weighted case, assign weight $w(x)$ to vertex x if $x \in S$ and $-w(x)$ if $x \notin S$. Find the path with the minimum number of edges among all minimum length paths and alternate it.

9.5 Min Plus Convolution [09b5c3]

```
// a is convex a[i+1]-a[i] <= a[i+2]-a[i+1]
vector<int> min_plus_convolution(vector<int> &a, vector
    <int> &b) {
  int n = SZ(a), m = SZ(b);
  vector<int> c(n + m - 1, INF);
  auto dc = [&](auto Y, int l, int r, int jl, int jr) {
    if (l > r) return;
    int mid = (l + r) / 2, from = -1, &best = c[mid];
    for (int j = jl; j <= jr; ++j)
      if (int i = mid - j; i >= 0 && i < n)
        if (best > a[i] + b[j])
          best = a[i] + b[j], from = j;
    Y(Y, l, mid - 1, jl, from), Y(Y, mid + 1, r, from,
        jr);
  };
  return dc(dc, 0, n - 1 + m - 1, 0, m - 1), c;
}
```

9.6 Mo's Algorithm [ea5261]

```
struct MoAlgorithm {
  struct query {
    int l, r, id;
    bool operator < (const query &o) {
      if (l / C == o.l / C)
        return (l / C) & 1 ? r > o.r : r < o.r;
      return l / C < o.l / C;
    }
  };
  int cur_ans;
  vector <int> ans;
  void add(int x) {} // do something
  void sub(int x) {} // do something
  vector <query> Q;
  void add_query(int l, int r, int id) { // [l, r)
    Q.push_back({l, r, id});
    ans.push_back(0);
  }
  void run() {
    sort(Q.begin(), Q.end());
    int pl = 0, pr = 0;
    cur_ans = 0;
    for (query &i : Q) {
      while (pl > i.l) add(a[--pl]);
      while (pr < i.r) add(a[pr++]);
      while (pl < i.l) sub(a[pl++]);
      while (pr > i.r) sub(a[--pr]);
      ans[i.id] = cur_ans;
    }
  }
};
```

9.7 Mo's Algorithm On Tree [8331c2]

```
/*  Mo's Algorithm On Tree
    Preprocess:
    1) LCA
    2) dfs with in[u] = dft++, out[u] = dft++
    3) ord[in[u]] = ord[out[u]] = u
    4) bitset<MAXN> inset                     */
struct Query {
  int L, R, LBid, lca;
  Query(int u, int v) {
    int c = LCA(u, v);
    if (c == u || c == v)
      q.lca = -1, q.L = out[c ^ u ^ v], q.R = out[c];
    else if (out[u] < in[v])
      q.lca = c, q.L = out[u], q.R = in[v];
    else
      q.lca = c, q.L = out[v], q.R = in[u];
    q.Lid = q.L / blk;
  }
  bool operator<(const Query &q) const {
```

```
    if (LBid != q.LBid) return LBid < q.LBid;
    return R < q.R;
  }
};
void flip(int x) {
    if (inset[x]) sub(arr[x]); // TODO
    else add(arr[x]); // TODO
    inset[x] = ~inset[x];
}
void solve(vector<Query> query) {
  sort(ALL(query));
  int L = 0, R = 0;
  for (auto q : query) {
    while (R < q.R) flip(ord[++R]);
    while (L > q.L) flip(ord[--L]);
    while (R > q.R) flip(ord[R--]);
    while (L < q.L) flip(ord[L++]);
    if (~q.lca) add(arr[q.lca]);
    // answer query
    if (~q.lca) sub(arr[q.lca]);
  }
}
```

9.8 PBDS [d65996]

```
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
tree<int, null_type, less<int>, rb_tree_tag,
     tree_order_statistics_node_update> oset;
// order_of_key, find_by_order
cc_hash_table<int, int> m1;
gp_hash_table<int, int> m2;
// like map, but much faster
```

9.9 Simulated Annealing [de78c6]

```
double factor = 100000;
const int base = 1e9; // remember to run ~ 10 times
for (int it = 1; it <= 1000000; ++it) {
  // ans: answer, nw: current value, rnd(): mt19937
  if (exp(-(nw - ans) / factor) >= (double)(rnd() %
       base) / base) ans = nw;
  factor *= 0.99995;
}
```

9.10 SOS dp [6aadb1]

```
//memory optimized, super easy to code.
rep(i, (1 << N)) F[i] = A[i];
rep(i, N) rep(mask, (1 << N)) {
  if(mask & (1<<i)) F[mask] += F[mask^(1<<i)];
}
```

9.11 SMAWK [a2a4ce]

```
// For all 2x2 submatrix:
// If M[1][0] < M[1][1], M[0][0] < M[0][1]
// If M[1][0] == M[1][1], M[0][0] <= M[0][1]
// M[i][ans_i] is the best value in the i-th row
// select(int r, int u, int v) return true if f(r, v)
     is better than f(r, u)
vector<int> smawk(int N, int M, auto &&select) {
  auto dc = [&](auto self, const vector<int> &r, const
       vector<int> &c) {
    if (r.empty()) return vector<int>{};
    const int n = SZ(r); vector<int> ans(n), nr, nc;
    for (int i : c) {
      while (!nc.empty() &&
          select(r[nc.size() - 1], nc.back(), i))
        nc.pop_back();
      if (int(nc.size()) < n) nc.push_back(i);
    }
    for (int i = 1; i < n; i += 2) nr.push_back(r[i]);
    const auto na = self(self, nr, nc);
    for (int i = 1; i < n; i += 2) ans[i] = na[i >> 1];
    for (int i = 0, j = 0; i < n; i += 2) {
      ans[i] = nc[j];
      const int end = i + 1 == n ? nc.back() : ans[i +
          1];
      while (nc[j] != end)
        if (select(r[i], ans[i], nc[++j])) ans[i] = nc[
            j];
    }
    return ans;
  };
  vector<int> R(N), C(M); iota(iter(R), 0), iota(iter(C
      ), 0);
  return dc(dc, R, C);
}
```

9.12 Tree Hash [34aae5]

```
ull seed;
ull shift(ull x) {
  x ^= x << 13;
  x ^= x >> 7;
  x ^= x << 17;
  return x;
}
ull dfs(int u, int f) {
  ull sum = seed;
  for (int i : G[u]) if (i != f)
    sum += shift(dfs(i, u));
  return sum;
}
```

9.13 Python [ebfb5e]

```
from [decimal, fractions, math, random] import *
setcontext(Context(prec=10, Emax=MAX_EMAX, rounding=
     ROUND_FLOOR))
Decimal('1.1') / Decimal('0.2')
Fraction(3, 7)
Fraction(Decimal('1.14'))
Fraction('1.2').limit_denominator(4).numerator
Fraction(cos(pi / 3)).limit_denominator()
print(*[randint(1, C) for i in range(0, N)], sep=' ')
```